

ArduPilot 飞控项目完整学习指南

本文档旨在帮助飞控开发初学者系统学习ArduPilot开源飞控项目，掌握飞控设计的框架、架构、模块功能及控制算法。

目录

- [1. 项目概述](#)
- [2. 整体架构](#)
- [3. 核心框架详解](#)
- [4. 传感器系统](#)
- [5. 姿态估计与导航](#)
- [6. 控制系统](#)
- [7. 参数系统](#)
- [8. 通信系统](#)
- [9. 飞行器类型](#)
- [10. 学习路径与实践建议](#)

1. 项目概述

1.1 什么是ArduPilot

ArduPilot是世界上最成熟的开源无人机飞控项目之一，由全球开发者社区维护，经过十余年的发展和完善。其代码质量高、架构清晰、文档完善，是学习飞控开发的最佳选择。

1.2 支持的飞行器类型

类型	目录	说明
多旋翼	ArduCopter/	四旋翼、六旋翼、八旋翼等
固定翼	ArduPlane/	常规飞机、VTOL倾转旋翼等
无人车	Rover/	无人驾驶车辆
水下机器人	ArduSub/	无人潜水器
天线追踪器	AntennaTracker/	追踪移动目标
浮空器	Blimp/	气球或飞艇

1.3 代码仓库结构

```
1 ardupilot/
2 |— ArduCopter/      # 多旋翼飞控代码（入门推荐从此开始）
3 |— ArduPlane/      # 固定翼飞控代码
4 |— ArduSub/        # 水下机器人代码
5 |— Rover/          # 无人车代码
6 |— AntennaTracker/ # 天线追踪器代码
7 |— Blimp/          # 浮空器代码
8 |— libraries/      # 核心库（最重要！理解架构的关键）
9 |— Tools/          # 工具集（SITL仿真、编译工具等）
10 |— modules/        # 外部依赖模块
11 |— docs/           # 官方文档
12 |— tests/          # 测试代码
13 |— waf / wscript   # 构建系统配置
```

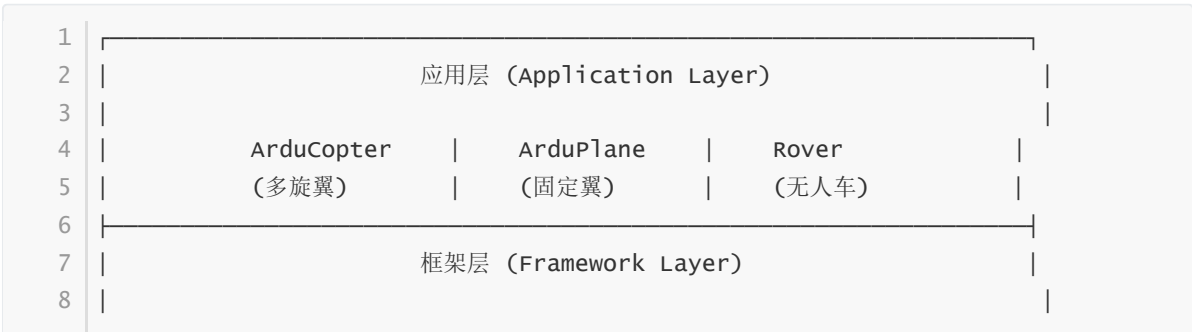
1.4 核心库目录详解

```
1 libraries/
2 |— AP_HAL/          # 【硬件抽象层】隔离硬件差异
3 |— AP_Common/       # 公共基础定义和工具
4 |— AP_Param/         # 【参数管理系统】配置的核心
5 |— AP_Scheduler/    # 【任务调度器】飞控的心跳
6 |— AP_Vehicle/      # 【飞控基类】所有飞行器的父类
7 |— AP_AHRS/         # 【姿态航向参考系统】姿态估计
8 |— AP_InertialSensor/ # IMU传感器驱动
9 |— AP_GPS/          # GPS接收机驱动
10 |— AP_Baro/         # 气压计驱动
11 |— AP_Compass/     # 磁力计驱动
12 |— AP_NavEKF3/      # 扩展卡尔曼滤波（位置/速度估计）
13 |— AC_AttitudeControl/ # 姿态控制器
14 |— AC_WPNav/        # 航点导航
15 |— GCS_MAVLink/     # MAVLink地面站通信
16 |— Filter/         # 数字滤波器
17 |— AP_Math/         # 数学库（向量、矩阵运算）
18 |— PID/            # PID控制器
19 |— ...              # 更多专用库
```

2. 整体架构

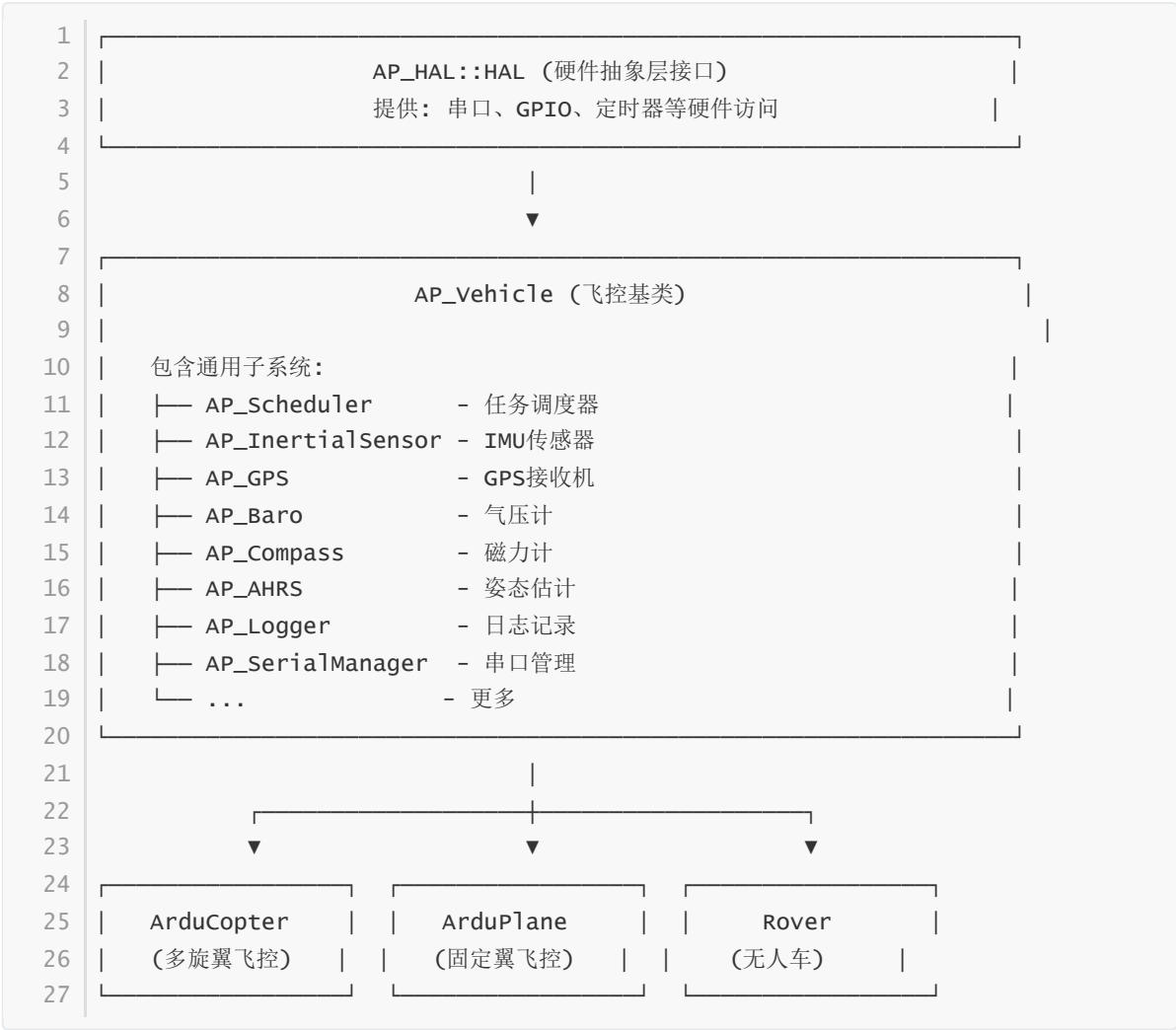
2.1 分层架构设计

ArduPilot采用经典的**分层架构**设计，这是企业级软件工程的最佳实践：





2.2 核心类继承关系



2.3 单例模式 (Singleton Pattern)

ArduPilot广泛使用**单例模式**来管理全局对象，确保整个系统只有一个实例：

```
1 // 全局单例访问宏（定义在各个库中）
2 AP::scheduler()      // 获取任务调度器实例
3 AP::ahrs()           // 获取姿态估计器实例
4 AP::gps()            // 获取GPS实例
5 AP::ins()            // 获取IMU传感器实例
6 AP::compass()        // 获取磁力计实例
7 AP::barometer()      // 获取气压计实例
8 AP::param()          // 获取参数系统实例
9 AP::vehicle()        // 获取飞控实例
10 AP::logger()         // 获取日志实例
11
12 // 使用示例
13 void my_function() {
14     // 获取当前姿态
15     float roll = AP::ahrs().get_roll_deg();
16     float pitch = AP::ahrs().get_pitch_deg();
17
18     // 获取GPS数据
19     if (AP::gps()->status() >= GPS_FIX_3D) {
20         float lat = AP::gps()->location().lat;
21     }
22
23     // 获取IMU数据
24     Vector3f gyro = AP::ins().get_gyro();
25 }
```

为什么要用单例？

- 整个系统只需要一个实例（传感器、调度器等）
- 方便全局访问，避免复杂的依赖注入
- 减少内存占用，简化代码

3. 核心框架详解

3.1 AP_Vehicle - 飞控基类

文件位置: `libraries/AP_Vehicle/AP_Vehicle.h`

AP_Vehicle是所有飞行器的**基类**，定义了通用接口和公共子系统。

3.1.1 核心成员变量详解

```
1 class AP_Vehicle : public AP_HAL::HAL::Callbacks {
2 protected:
3     // ===== 【任务调度器】 =====
4     // 飞控的心跳系统，决定每个任务何时执行
5     AP_Scheduler scheduler;
6
7     // ===== 【传感器系统】 =====
```

```

8 // GPS接收机 - 提供位置、速度、时间
9 AP_GPS gps;
10
11 // 气压计 - 提供高度估计
12 AP_Baro barometer;
13
14 // 磁力计 - 提供航向角
15 Compass compass;
16
17 // IMU传感器 - 提供加速度和角速度
18 AP_InertialSensor ins;
19
20 // 按钮 - 物理按钮输入
21 AP_Button button;
22
23 // 距离传感器 - 测距（定高、避障）
24 RangeFinder rangefinder;
25
26 // ===== 【状态估计】 =====
27 // 姿态航向参考系统 - 融合传感器数据，估计飞行器状态
28 AP_AHRS ahrs;
29
30 // ===== 【通信】 =====
31 // 串口管理器 - 管理多个串口
32 AP_SerialManager serial_manager;
33
34 // CAN总线管理器 - 连接CAN设备（电调、传感器等）
35 AP_CANManager can_mgr;
36
37 // ===== 【通知】 =====
38 // LED、蜂鸣器等状态指示
39 AP_Notify notify;
40
41 // ===== 【日志】 =====
42 // 飞行数据记录
43 AP_Logger logger;
44
45 // ===== 【时间管理】 =====
46 // 积分时间 - 上一帧循环耗时，用于积分计算
47 float G_Dt;
48 };

```

3.1.2 核心方法详解

```

1 class AP_Vehicle : public AP_HAL::HAL::Callbacks {
2 public:
3     /**
4      * 初始化入口 - HAL层调用
5      * 注意：使用final关键字防止子类覆盖，强制使用init_ardupilot()
6      */
7     void setup(void) override final;
8
9     /**
10      * 主循环 - HAL层调用
11      * 内部调用scheduler.loop()执行任务调度

```

```

12     */
13     void loop() override final;
14
15     // ===== 虚方法（由子类实现） =====
16
17     /**
18      * 子类特定的初始化
19      * ArduCopter实现：初始化电机、传感器、飞行模式等
20      */
21     virtual void init_ardupilot() = 0;
22
23     /**
24      * 加载参数
25      * 从EEPROM加载保存的配置
26      */
27     virtual void load_parameters() = 0;
28
29     /**
30      * 设置飞行模式
31      * @param new_mode 新的模式编号
32      * @param reason 模式切换原因
33      * @return 是否切换成功
34      */
35     virtual bool set_mode(const uint8_t new_mode, const ModeReason reason) =
36 0;
37
38     /**
39      * 获取当前飞行模式
40      */
41     virtual uint8_t get_mode() const = 0;
42 };

```

设计意图说明：

- `setup()` 和 `loop()` 是HAL层调用的入口
- `final` 关键字强制子类使用 `init_ardupilot()` 和 `load_parameters()` 进行定制化初始化
- 所有传感器、子系统都在AP_Vehicle中声明，子类直接继承使用

3.2 AP_Scheduler - 任务调度器

文件位置: `libraries/AP_Scheduler/AP_Scheduler.h`

任务调度器是飞控的“心跳”，决定每个任务在何时执行。这是实时系统的核心组件。

3.2.1 任务定义结构

```

1 struct Task {
2     task_fn_t function;           // 任务函数指针（void func(void)类型）
3     const char *name;             // 任务名称（用于日志和调试）
4     float rate_hz;                // 运行频率（Hz），0表示最快频率
5     uint16_t max_time_micros;     // 最大允许执行时间（微秒），用于性能监控
6     uint8_t priority;             // 优先级，数字越大优先级越高
7 };
8
9 /**

```

```

10  * 任务定义宏 - 用于简化任务定义
11  *
12  * @param classname 所属的类名
13  * @param classptr 类的实例指针
14  * @param func 要调用的方法名
15  * @param _rate_hz 执行频率 (Hz)
16  * @param _max_time_micros 最大执行时间 (微秒)
17  * @param _priority 优先级
18  *
19  * 使用示例:
20  * SCHED_TASK_CLASS(Copter, &copter, update_GPS, 50, 100, 0)
21  * 表示: copter.update_GPS() 每20ms调用一次, 最大执行100微秒
22  */
23 #define SCHED_TASK_CLASS(classname, classptr, func, _rate_hz,
    _max_time_micros, _priority) { \
24     .function = FUNCTOR_BIND(classptr, &classname::func, void), \
25     AP_SCHEDULER_NAME_INITIALIZER(classname, func) \
26     .rate_hz = _rate_hz, \
27     .max_time_micros = _max_time_micros, \
28     .priority = _priority \
29 }

```

3.2.2 多旋翼任务列表示例

```

1  // 文件: ArduCopter/Copter.cpp
2
3  // 任务定义表 - 定义所有需要在循环中执行的任务
4  const AP_Scheduler::Task Copter::scheduler_tasks[] = {
5      // 【快速任务】最高优先级, 每帧执行
6      { &Copter::fast_loop, "fast_loop", 0, 0, 0 },
7
8      // 【GPS任务】50Hz - 每20ms执行一次
9      { &Copter::update_GPS, "GPS", 50, 100, 0 },
10
11     // 【AHRS任务】50Hz - 更新姿态估计
12     { &Copter::update_ahrs, "AHRS", 50, 100, 0 },
13
14     // 【悬停油门】50Hz - 计算悬停油门
15     { &Copter::update_throttle_hover, "ThrottleHover", 50, 100, 0 },
16
17     // 【电池罗盘】10Hz - 每100ms执行
18     { &Copter::update_batt_compass, "BattCompass", 10, 100, 0 },
19
20     // 【1Hz任务】每秒执行一次
21     { &Copter::one_hz_loop, "OneHz", 1, 100, 0 },
22
23     // 【飞控模式更新】50Hz
24     { &Copter::update_flight_mode, "FlightMode", 50, 100, 0 },
25
26     // 【日志记录】50Hz
27     { &Copter::ten_hz_logging_loop, "Log", 50, 200, 0 },
28 };
29
30 /**
31  * 频率计算规则:

```

```

32 * - 基础循环频率 = 400Hz (多旋翼默认)
33 * - rate_hz = 50 意味着每 400/50 = 8 个tick执行一次
34 * - rate_hz = 10 意味着每 400/10 = 40 个tick执行一次
35 * - rate_hz = 1 意味着每 400/1 = 400 个tick执行一次
36 */

```

3.2.3 fast_loop - 核心循环

```

1 /**
2  * fast_loop() - 最高频率任务 (400Hz)
3  * 这是飞控最关键的任务，包含姿态控制和安全检查
4  */
5 void Copter::fast_loop() {
6     // ===== 1. 读取传感器 =====
7     // 读取IMU数据 (加速度计、陀螺仪)
8     read_inertia();
9
10    // 读取GPS数据
11    update_GPS();
12
13    // 读取气压计
14    read_barometer();
15
16    // 读取距离传感器
17    read_rangefinder();
18
19    // ===== 2. 更新状态估计 =====
20    // AHRS使用IMU数据更新姿态估计
21    update_AHRS();
22
23    // ===== 3. 更新飞行模式 =====
24    // 根据当前模式处理输入和输出
25    update_flight_mode();
26
27    // ===== 4. 执行控制 =====
28    // 角速度控制器 - 计算电机输出
29    attitude_control->rate_controller_run();
30
31    // ===== 5. 输出到电机 =====
32    motors_output();
33 }
34
35 /**
36  * one_hz_loop() - 低频率任务 (1Hz)
37  * 执行不需要高频率更新的任务
38  */
39 void Copter::one_hz_loop() {
40     // 电池状态检查
41     battery.check();
42
43     // GPS状态检查
44     gps.check();
45
46     // 更新LED状态
47     notify.update();

```



```

48
49 // 保存日志
50 Log_Write_Data(LogDataID::AP_STATE, 0);
51 }

```

3.2.4 调度器核心方法

```

1  class AP_Scheduler {
2  public:
3      /**
4       * 初始化调度器
5       * @param tasks      任务数组
6       * @param num_tasks   任务数量
7       * @param log_performance_bit 性能日志位掩码
8       */
9      void init(const Task *tasks, uint8_t num_tasks, uint32_t
log_performance_bit);
10
11     /**
12      * 主循环调用 - 执行任务调度
13      * 这是被HAL.loop()调用的核心函数
14      */
15     void loop();
16
17     /**
18      * 时钟tick - 更新时钟计数器
19      * 每次循环调用一次
20      */
21     void tick(void);
22
23     /**
24      * 运行任务
25      * @param time_available 本次循环可用时间（微秒）
26      */
27     void run(uint32_t time_available);
28
29     /**
30      * 获取当前循环频率（Hz）
31      * 多旋翼默认400Hz
32      */
33     uint16_t get_loop_rate_hz(void) {
34         return _active_loop_rate_hz;
35     }
36
37     /**
38      * 获取循环负载（0.0-1.0之间）
39      * 1.0表示100%负载，CPU满负荷运行
40      */
41     float load_average();
42
43     /**
44      * 获取最后循环耗时（秒）
45      */
46     float get_last_loop_time_s(void) const {
47         return _last_loop_time_s;

```

```
48     }
49 };
```

3.2.5 调度原理图解

主循环流程:

```
graph TD; A[HAL.main_loop() 被定时器调用] --> B["1. scheduler.tick() - 更新时钟  
_tick_counter++  
_loop_sample_time_us = 当前时间"]; B --> C["2. 计算可用时间  
loop_period = 1秒 / 400Hz = 2500微秒  
available = loop_period - 上一帧耗时"]; C --> D["3. scheduler.run(available) - 执行到期的任务  
  
For 每个任务:  
    if (任务应该执行) {  
        记录开始时间  
        执行任务函数  
        记录结束时间  
        如果超时, 标记警告  
    }"]; D --> E["4. 计算负载并记录性能  
_last_loop_time_s = 实际耗时  
load = 1.0 - spare_time / loop_period"];
```

任务执行判断示例:

- 基础频率 = 400Hz
- 任务A: rate_hz = 400 → 每1个tick执行一次 → 每2.5ms
- 任务B: rate_hz = 200 → 每2个tick执行一次 → 每5ms
- 任务C: rate_hz = 50 → 每8个tick执行一次 → 每20ms
- 任务D: rate_hz = 10 → 每40个tick执行一次 → 每100ms

为什么要这样设计？

1. **确定性**: 高优先级任务（姿态控制）总是优先执行
2. **负载均衡**: 自动根据CPU负载调整任务执行

- 3. **可监控**：记录每个任务的执行时间，便于调试
- 4. **可配置**：可以通过参数调整循环频率

3.3 HAL (Hardware Abstraction Layer) - 硬件抽象层

文件位置: `libraries/AP_HAL/`

HAL层将硬件操作抽象成**统一接口**，使上层代码与具体硬件解耦。

3.3.1 HAL类结构

```
1 // 文件: libraries/AP_HAL/HAL.h
2
3 class HAL {
4 public:
5     /**
6      * 获取串口驱动
7      * @param num 串口号 (0, 1, 2, ...)
8      * @return 串口驱动指针
9      */
10    UARTDriver *serial(int num);
11
12    // ===== 通用硬件接口 =====
13    GPIO *gpio;                // GPIO引脚控制
14    RTC *rtc;                  // 实时时钟
15    System *system;           // 系统控制
16    Storage *storage;          // 持久化存储
17
18    // ===== 模拟/数字 =====
19    AnalogIn *analogin;        // 模拟输入（电池电压检测）
20
21    // ===== RC输入输出 =====
22    RCOutput *rcout;           // PWM输出（电机、舵机）
23    RCInput *rcin;             // PWM输入（接收机）
24
25    // ===== 调度 =====
26    Scheduler *scheduler;      // 任务调度器
27
28    // ===== 示波器（调试） =====
29    void (*console_backend)(AP_HAL::BetterStream *);
30 };
```

3.3.2 HAL实现类

实现类	平台	用途
<code>AP_HAL_ChibiOS::HAL</code>	STM32	主要嵌入式飞控平台
<code>AP_HAL_SITL::HAL</code>	Linux/Windows	软件在环仿真
<code>AP_HAL_Linux::HAL</code>	Linux	Linux原生运行
<code>AP_HAL_ESP32::HAL</code>	ESP32	ESP32飞控

3.3.3 全局HAL访问

```
1 // 全局HAL对象声明（由各平台HAL实现提供）
2 extern const AP_HAL::HAL& hal;
3
4 // ===== 常用操作示例 =====
5
6 // 1. 串口输出
7 void uart_example() {
8     AP_HAL::UARTDriver *uart = hal.serial(0); // 串口0通常是USB/调试串口
9     uart->printf("Hello, World!\n");
10    uart->printf("Value: %d, Float: %.2f\n", 123, 45.67);
11 }
12
13 // 2. GPIO控制
14 void gpio_example() {
15     // 设置引脚模式
16     hal.gpio->pinMode(13, OUTPUT); // LED引脚
17
18     // 写引脚
19     hal.gpio->digitalWrite(13, HIGH); // LED亮
20     hal.gpio->digitalWrite(13, LOW); // LED灭
21
22     // 读引脚
23     bool value = hal.gpio->digitalRead(12);
24 }
25
26 // 3. PWM输出（电机控制）
27 void pwm_example() {
28     // 启用通道
29     hal.rcout->enable_ch(0); // 通道0
30     hal.rcout->enable_ch(1); // 通道1
31
32     // 设置PWM值（1000-2000µs）
33     hal.rcout->write(0, 1500); // 通道0输出1500µs
34     hal.rcout->write(1, 1000); // 通道1输出1000µs
35 }
36
37 // 4. PWM输入（接收机）
38 void pwm_input_example() {
39     uint16_t channels[8];
40     hal.rcin->read(channels, 8); // 读取8个通道
41     uint16_t throttle = channels[2]; // 通常通道2是油门
42 }
43
44 // 5. 延时
45 void delay_example() {
46     hal.scheduler->delay(100); // 阻塞延时100ms
47     hal.scheduler->delay_microseconds(100); // 阻塞延时100µs
48 }
```

3.3.4 时间函数

```
1 // ===== 时间获取 =====
2
3 // 毫秒计数器
4 uint32_t ms = AP_HAL::millis(); // 32位毫秒（约49天溢出）
5 uint64_t ms64 = AP_HAL::millis64(); // 64位毫秒（永不过期）
6
7 // 微秒计数器
8 uint32_t us = AP_HAL::micros(); // 32位微秒（约71分钟溢出）
9 uint64_t us64 = AP_HAL::micros64(); // 64位微秒（永不过期）
10
11 // ===== 时间使用示例 =====
12
13 // 1. 计算时间差
14 uint32_t start_time = AP_HAL::millis();
15 do_something();
16 uint32_t elapsed = AP_HAL::millis() - start_time;
17
18 // 2. 限制定时执行
19 static uint32_t last_print_ms = 0;
20 if (AP_HAL::millis() - last_print_ms > 1000) { // 每秒执行一次
21     last_print_ms = AP_HAL::millis();
22     // 执行操作
23 }
24
25 // 3. 非阻塞延时（在循环中使用）
26 #define PRINT_INTERVAL 1000 // 1000ms
27 uint32_t last_print = 0;
28
29 void loop() {
30     uint32_t now = AP_HAL::millis();
31     if (now - last_print >= PRINT_INTERVAL) {
32         last_print = now;
33         // 每秒执行一次的操作
34     }
35 }
```

HAL设计的好处：

1. **硬件无关**：同一套代码可以运行在STM32、ESP32、Linux等不同平台
2. **易于移植**：新硬件支持只需实现HAL接口
3. **便于测试**：可以在SITL仿真中测试所有功能逻辑

4. 传感器系统

4.1 AP_InertialSensor - IMU传感器

文件位置：libraries/AP_InertialSensor/

IMU（Inertial Measurement Unit，惯性测量单元）是飞控的"感觉器官"，包含**加速度计**和**陀螺仪**。

4.1.1 传感器坐标系

```
1  传感器坐标系（右手定则）：
2      z（向上）
3      ↑
4      |
5      |→ y（向右）
6      /
7     /
8    ↗ x（向前）
9
10 对于安装在无人机上的IMU：
11  - x轴：指向机头方向
12  - y轴：指向机体右侧
13  - z轴：指向机体下方（通常朝下）
14
15 注意：坐标系方向取决于传感器安装方向
```

4.1.2 IMU类结构

```
1  class AP_InertialSensor {
2  public:
3      // ===== 数据获取方法 =====
4
5      /**
6       * 获取陀螺仪数据（原始/未滤波）
7       * @param instance IMU实例编号（0, 1, 2...）
8       * @return 角速度向量，单位：弧度/秒
9       *
10      * gyro.x = 绕X轴旋转速度（Roll rate）
11      * gyro.y = 绕Y轴旋转速度（Pitch rate）
12      * gyro.z = 绕Z轴旋转速度（Yaw rate）
13      */
14      const Vector3f& get_gyro(uint8_t instance=0) const;
15
16      /**
17       * 获取加速度计数据
18       * @param instance IMU实例编号
19       * @return 加速度向量，单位：m/s²
20       *
21       * accel.z ≈ 9.8 表示水平放置
22       * accel.x/y ≈ 0 表示水平放置
23       */
24      const Vector3f& get_accel(uint8_t instance=0) const;
25
26      /**
27       * 检测是否新的陀螺仪偏移数据
28       */
29      bool get_new_gyro_offsets(Vector3f &offsets) const;
30
31      /**
32       * 获取传感器温度（用于温度补偿）
33       */
34      float get_temperature(uint8_t instance=0) const;
```

```

35
36     /**
37      * 检查IMU是否健康
38      */
39     bool healthy(uint8_t instance=0) const;
40
41     /**
42      * 获取样本计数（用于同步）
43      */
44     uint32_t get_sample_count() const { return _sample_count; }
45
46     /**
47      * 获取样本时间戳（微秒）
48      */
49     uint64_t get_sample_time_us(uint8_t instance=0) const;
50 };

```

4.1.3 IMU使用示例

```

1 // ===== 1. 获取IMU数据 =====
2
3 void read_imu() {
4     // 获取主IMU（实例0）
5     Vector3f gyro = AP::ins().get_gyro();
6     Vector3f accel = AP::ins().get_accel();
7
8     // 打印数据（调试用）
9     hal.console->printf("Gyro: %.2f, %.2f, %.2f rad/s\n",
10                        degrees(gyro.x), degrees(gyro.y), degrees(gyro.z));
11     hal.console->printf("Accel: %.2f, %.2f, %.2f m/s²\n",
12                        accel.x, accel.y, accel.z);
13
14     // 检查健康状态
15     if (!AP::ins().healthy()) {
16         // IMU不健康，需要处理
17     }
18 }
19
20 // ===== 2. 多IMU冗余 =====
21
22 void multi_imu_example() {
23     uint8_t num_imus = AP::ins().get_num_instances();
24
25     for (uint8_t i = 0; i < num_imus; i++) {
26         Vector3f gyro = AP::ins().get_gyro(i);
27         Vector3f accel = AP::ins().get_accel(i);
28
29         // 检查每个IMU的健康状态
30         if (AP::ins().healthy(i)) {
31             // 使用该IMU数据
32         }
33     }
34 }
35
36 // ===== 3. 检测新数据 =====

```

```
37
38 bool new_data_available() {
39     static uint32_t last_sample_count = 0;
40     uint32_t current_count = AP::ins().get_sample_count();
41
42     if (current_count != last_sample_count) {
43         last_sample_count = current_count;
44         return true;    // 有新数据
45     }
46     return false;
47 }
```

4.1.4 支持的IMU芯片

芯片型号	厂商	量程	说明
MPU6000	Invensense	±2000°/s, ±16g	经典6轴IMU，最常用
MPU6500	Invensense	±2000°/s, ±16g	升级版MPU6000
MPU9250	Invensense	±2000°/s, ±16g	9轴IMU（集成磁力计）
ICM20649	Invensense	±4000°/s, ±30g	高性能，运动追踪
ICM20689	Invensense	±4000°/s, ±30g	飞行专用
BMI055	Bosch	±2000°/s, ±16g	汽车级传感器
LSM9DS1	ST	±2000°/s, ±16g	9轴IMU

4.1.5 IMU数据处理流程

1 传感器数据处理流程：

2

3 | 1. 硬件读取阶段 |

4 | ├─ 通过SPI/I2C接口读取芯片寄存器 |

5 | ├─ 陀螺仪数据单位：rad/s |

6 | └─ 加速度计数据单位：m/s² |

7 |_____|

8 | 2. 温度补偿阶段 |

9 | ├─ 传感器对温度敏感（温漂） |

10 | ├─ 使用内置温度传感器校准 |

11 | └─ 公式：校正值 = 原始值 - 温度补偿系数 × 温度差 |

12 |_____|

13 | 3. 零偏校准阶段 |

14 | ├─ 静止时陀螺仪应有0输出 |

15 | ├─ 实际存在零点偏移 |

16 | └─ 公式：校正值 = 原始值 - 零偏 |

17 |_____|

18 | 4. 滤波降噪阶段 |

19 | ├─ 低通滤波器去除高频噪声 |

20 | ├─ 采样频率 >> 截止频率（通常>10x） |

21 | └─ 公式：y[n] = α × x[n] + (1-α) × y[n-1] |

22 |_____|

23 | 5. EKF融合阶段 |


```

24 | └─ 与GPS、磁力计等传感器数据融合
25 | └─ 输出：姿态角、位置、速度
26 |

```

4.2 AP_GPS - GPS接收机

文件位置: `libraries/AP_GPS/`

GPS提供位置、速度和时间信息，是室外飞行的必需传感器。

4.2.1 GPS数据类结构

```

1 // Location类 - 地理位置表示
2 class Location {
3 public:
4     int32_t lat;        // 纬度 × 10^7 (度)
5     int32_t lng;        // 经度 × 10^7 (度)
6     int32_t alt;        // 高度 (厘米)，相对于家点
7     int32_t relative_alt; // 相对高度
8     int32_t ground_course; // 地面航向 (0.01度)
9     uint16_t ground_speed; // 地面速度 (cm/s)
10
11     // 转换为度
12     double latitude_deg() const { return lat / 1.0e7; }
13     double longitude_deg() const { return lng / 1.0e7; }
14     float alt_m() const { return alt / 100.0f; }
15 };

```

4.2.2 GPS类方法

```

1 class AP_GPS {
2 public:
3     /**
4      * GPS定位状态枚举
5      */
6     enum GPSStatus {
7         NO_FIX = 0,        // 无定位，无法使用
8         FIX_2D = 1,        // 2D定位 (只有水平位置)
9         FIX_3D = 2,        // 3D定位 (有高度)
10        FIX_3D_DGPS = 3,    // 3D + 差分GPS (DGPS)
11        FIX_3D_RTK = 4,     // 3D + RTK (厘米级精度)
12    };
13
14    /**
15     * 获取GPS状态
16     */
17    GPSStatus status(uint8_t instance=0) const;
18
19    /**
20     * 获取地理位置
21     */
22    const Location& location(uint8_t instance=0) const;
23
24    /**

```

```

25     * 获取速度向量 (NED坐标系)
26     * @return 速度向量, 单位: m/s
27     */
28     const Vector3f& velocity(uint8_t instance=0) const;
29
30     /**
31     * 获取水平速度精度 (HDOP)
32     * 值越小精度越高
33     */
34     float get_hdop(uint8_t instance=0) const;
35
36     /**
37     * 获取定位精度
38     * @return 精度估计 (米)
39     */
40     float get_accuracy(uint8_t instance=0) const;
41
42     /**
43     * 获取GPS时间
44     */
45     uint64_t time_week_ms(uint8_t instance=0) const;
46     uint16_t week(uint8_t instance=0) const;
47
48     /**
49     * 获取水平速度
50     */
51     float ground_speed(uint8_t instance=0) const;
52
53     /**
54     * 获取航向角 (地面航向)
55     */
56     float ground_course(uint8_t instance=0) const;
57
58     /**
59     * 获取可见卫星数量
60     */
61     uint8_t num_sats(uint8_t instance=0) const;
62 };

```

4.2.3 GPS使用示例

```

1  // ===== 1. 基本使用 =====
2
3  void gps_example() {
4      AP_GPS *gps = AP::gps();
5
6      // 检查GPS状态
7      GPSStatus fix_status = gps->status();
8
9      if (fix_status >= GPS_FIX_3D) {
10         // GPS定位有效
11         const Location &loc = gps->location();
12
13         float lat = loc.lat / 1.0e7;          // 转换为度
14         float lon = loc.lng / 1.0e7;

```

```

15     float alt = loc.alt / 100.0f;        // 转换为米
16
17     hal.console->printf("GPS: %.7f, %.7f, %.1fm\n", lat, lon, alt);
18
19     // 获取速度
20     Vector3f vel = gps->velocity();
21     float speed = vel.length();          // 水平速度 m/s
22     float climb_rate = -vel.z;           // 上升率 m/s
23
24     // 获取精度
25     float accuracy = gps->get_accuracy();
26     uint8_t satellites = gps->num_sats();
27
28     hal.console->printf("Speed: %.1f m/s, Sats: %d, Accuracy: %.1fm\n",
29                         speed, satellites, accuracy);
30 } else {
31     // GPS未定位
32     hal.console->printf("GPS not fixed: %d\n", (int)fix_status);
33 }
34 }
35
36 // ===== 2. GPS时间同步 =====
37
38 void gps_time_example() {
39     AP_GPS *gps = AP::gps();
40
41     if (gps->status() >= GPS_FIX_3D) {
42         uint64_t time_ms = gps->time_week_ms();
43         uint16_t week = gps->week();
44
45         // GPS周秒转换为UNIX时间戳（需要查表或计算GPS起始时间）
46         // GPS起始时间：1980年1月6日 00:00:00 UTC
47
48         hal.console->printf("GPS Time: Week %u, %llu ms\n",
49                             (unsigned)week, (unsigned long long)time_ms);
50     }
51 }
52
53 // ===== 3. 多GPS冗余 =====
54
55 void multi_gps_example() {
56     AP_GPS *gps = AP::gps();
57     uint8_t num_gps = gps->num_sensors();
58
59     for (uint8_t i = 0; i < num_gps; i++) {
60         if (gps->status(i) >= GPS_FIX_3D) {
61             const Location &loc = gps->location(i);
62             // 使用第i个GPS
63         }
64     }
65 }

```

4.2.4 支持的GPS协议

协议	说明	常见模块
u-blox	最常用，支持多星系统	u-blox M8N, M9N, F9P
NMEA	标准NMEA0183协议	通用GPS
SBF	Septentrio格式	Septentrio
SBP	SwiftNav SBP	Piksi
DroneCAN	UAVCAN/CAN总线GPS	CUAV CAN GPS
MAVLink	通过MAVLink获取	外部定位系统

4.2.5 GPS数据精度对比

定位类型	精度	用途
标准GPS	2-5m	基础飞行
SBAS增强	1-2m	航拍
DGPS	0.5-1m	测绘
RTK	1-2cm	精准农业、测绘

4.3 AP_Baro - 气压计

文件位置: `libraries/AP_Baro/`

气压计测量大气压力，通过气压-高度公式转换为高度。

4.3.1 气压高度原理

```
1  大气压力与高度的关系（简化模型）：  
2  P = P0 × (1 - 0.0065 × h / T0)^(5.257)  
3  
4  其中：  
5  - P = 测量到的气压（Pa）  
6  - P0 = 海平面气压（101325 Pa）  
7  - h = 高度（m）  
8  - T0 = 海平面温度（288.15 K）  
9  - 0.0065 = 温度递减率（K/m）  
10  
11 简化公式（低高度）：  
12 h ≈ 44330 × (1 - (P/P0)^0.1903)  
13  
14 ArduPilot使用查表法计算，精度更高
```

4.3.2 气压计类方法

```
1 class AP_Baro {
2 public:
3     /**
4      * 获取压力
5      * @return 压力值, 单位: 帕斯卡 (Pa)
6      */
7     float get_pressure(uint8_t instance=0) const;
8
9     /**
10    * 获取温度
11    * @return 温度值, 单位: 摄氏度
12    */
13    float get_temperature(uint8_t instance=0) const;
14
15    /**
16    * 获取相对于地面的高度
17    * @return 高度值, 单位: 米
18    */
19    float get_altitude(uint8_t instance=0) const;
20
21    /**
22    * 获取海拔高度 (绝对高度)
23    * @param ground_pressure 地面气压
24    * @return 海拔高度 (米)
25    */
26    float get_altitude_difference(uint8_t instance=0, float
ground_pressure=101325.0f) const;
27
28    /**
29    * 获取滤波后的高度 (推荐使用)
30    */
31    float get_filtered_altitude(uint8_t instance=0) const;
32 };
```

4.3.3 使用示例

```
1 void barometer_example() {
2     AP_Baro *baro = AP::barometer();
3
4     // 获取数据
5     float pressure = baro->get_pressure();        // Pa
6     float temperature = baro->get_temperature();  // °C
7     float altitude = baro->get_altitude();        // m
8
9     hal.console->printf("Baro: %.0f Pa, %.1f°C, %.1fm\n",
10                        pressure, temperature, altitude);
11
12    // 高度变化率 (爬升率)
13    static float last_alt = 0;
14    static uint32_t last_time = 0;
15
16    uint32_t now = AP_HAL::millis();
```

```
17     if (now != last_time) {
18         float climb_rate = (altitude - last_alt) / ((now - last_time) /
1000.0f);
19         last_alt = altitude;
20         last_time = now;
21
22         hal.console->printf("Climb rate: %.2f m/s\n", climb_rate);
23     }
24 }
```

4.3.4 支持的气压计芯片

芯片型号	精度	说明
MS5611	10cm	高精度，最常用
BMP280	1m	低成本
BMP388	50cm	替代BMP280
ICM20789	-	集成在ICM IMU中
DPS280	30cm	Bosch新一代

4.4 AP_Compass - 磁力计

文件位置: `libraries/AP_Compass/`

磁力计测量地球磁场，提供航向角（偏航角）。

4.4.1 地球磁场

1 地球磁场矢量：
2 北
3 ↑
4 | ← 磁场水平分量
5 |
6 ├───┐
7 | |
8 └───┴───→ 东
9 | |
10 ├───┐
11 | |
12 地球磁场强度：约25-65微特斯拉（μT）
13 磁场方向：水平分量指向磁北（与真北有磁偏角）

4.4.2 磁力计类方法

```
1 class Compass {
2 public:
3     /**
4      * 获取磁场数据（原始）
5      * @return 磁场向量，单位：微特斯拉（μT）
6      */
7     const Vector3f& get_field(uint8_t instance=0) const;
```

```

8
9  /**
10   * 获取磁场强度（标量）
11   */
12   float get_length(uint8_t instance=0) const;
13
14   /**
15   * 获取航向角
16   * @return 航向角（0-360度），0=北，90=东
17   */
18   float get_heading(uint8_t instance=0) const;
19
20   /**
21   * 获取校准后的磁场数据
22   */
23   const Vector3f& get_field_calibrated(uint8_t instance=0) const;
24
25   /**
26   * 磁力计是否健康
27   */
28   bool healthy(uint8_t instance=0) const;
29
30   /**
31   * 是否使用磁力计
32   */
33   bool use_for_yaw(uint8_t instance=0) const;
34 };

```

4.4.3 使用示例

```

1 void compass_example() {
2     Compass *compass = AP::compass();
3
4     // 获取磁场数据
5     Vector3f mag = compass->get_field();
6     float heading = compass->get_heading();
7
8     hal.console->printf("Mag: %.1f, %.1f, %.1f μT\n",
9                        mag.x, mag.y, mag.z);
10    hal.console->printf("Heading: %.1f°\n", heading);
11
12    // 磁偏角校正
13    // 获取磁场强度
14    float strength = compass->get_length();
15    hal.console->printf("Field strength: %.1f μT\n", strength);
16 }

```

4.4.4 磁力计校准

```
1  /**
2   * 磁力计校准是必需的！
3   * 因为：
4   * 1. 传感器存在零偏
5   * 2. 周围金属/磁场干扰
6   * 3. 传感器安装角度误差
7   *
8   * 校准方法：
9   * 1. 水平旋转飞行器360度
10  * 2. 倾斜飞行器，记录各角度数据
11  * 3. 拟合椭圆球体，找到中心偏移和缩放
12  * 4. 应用校准参数
13  */
```

5. 姿态估计与导航

5.1 AP_AHRS - 姿态航向参考系统

文件位置: `libraries/AP_AHRS/`

AHRS是飞控的“大脑”，融合所有传感器数据，估计飞行器的**姿态**和**位置**。

5.1.1 AHRS类结构

```
1  class AP_AHRS {
2  public:
3      // ===== 姿态角获取 =====
4
5      /**
6       * 获取欧拉角（弧度）
7       */
8      float get_roll_rad() const { return roll; }
9      float get_pitch_rad() const { return pitch; }
10     float get_yaw_rad() const { return yaw; }
11
12     /**
13      * 获取欧拉角（度）- 最常用
14      */
15     float get_roll_deg() const { return rpy_deg[0]; }
16     float get_pitch_deg() const { return rpy_deg[1]; }
17     float get_yaw_deg() const { return rpy_deg[2]; }
18
19     /**
20      * 获取传感器三角函数值（用于快速计算）
21      */
22     float cos_roll() const { return _cos_roll; }
23     float cos_pitch() const { return _cos_pitch; }
24     float cos_yaw() const { return _cos_yaw; }
25     float sin_roll() const { return _sin_roll; }
26     float sin_pitch() const { return _sin_pitch; }
27     float sin_yaw() const { return _sin_yaw; }
```



```

28
29 // ===== 旋转矩阵和四元数 =====
30
31 /**
32  * 获取旋转矩阵 (NED → 机体)
33  */
34 const Matrix3f& get_rotation_body_to_ned(void) const;
35
36 /**
37  * 获取四元数 (推荐, 优于欧拉角)
38  */
39 void get_quat_body_to_ned(Quaternion &quat) const;
40
41 // ===== 传感器数据 =====
42
43 /**
44  * 获取校正后的陀螺仪数据
45  */
46 const Vector3f& get_gyro(void) const { return state.gyro_estimate; }
47
48 /**
49  * 获取地球坐标系下的加速度
50  */
51 const Vector3f& get_accel_ef() const { return state.accel_ef; }
52
53 /**
54  * 获取陀螺仪漂移
55  */
56 const Vector3f& get_gyro_drift(void) const { return state.gyro_drift; }
57
58 // ===== 位置和速度 =====
59
60 /**
61  * 获取当前位置
62  */
63 bool get_location(Location &loc) const;
64
65 /**
66  * 获取NED速度
67  */
68 bool get_velocity_NED(Vector3f &vec) const;
69
70 /**
71  * 获取相对于家的位置
72  */
73 bool get_relative_position_NED_home(Vector3f &vec) const;
74
75 // ===== 风估计 =====
76
77 /**
78  * 获取风速估计
79  */
80 const Vector3f& wind_estimate() const { return state.wind_estimate; }
81 };

```

5.1.2 欧拉角详解

```
1 欧拉角定义（Z-Y-X顺序，航空常用）：
2
3  Roll（横滚角） $\phi$ ：
4  - 绕X轴旋转
5  - 正值：右翼向下
6  - 范围： $-180^\circ$  到  $180^\circ$ 
7
8  Pitch（俯仰角） $\theta$ ：
9  - 绕Y轴旋转
10 - 正值：机头向上
11 - 范围： $-90^\circ$  到  $90^\circ$ 
12
13 Yaw（偏航角） $\psi$ ：
14 - 绕Z轴旋转
15 - 正值：机头向左（顺时针看）
16 - 范围： $-180^\circ$  到  $180^\circ$ 
17
18 坐标系约定（NED）：
19 北（X）→ 前
20 东（Y）→ 右
21 下（Z）→ 上方为负
```

5.1.3 AHRS使用示例

```
1 // ===== 1. 获取姿态数据 =====
2
3 void ahrs_example() {
4     AP_AHRS &ahrs = AP::ahrs();
5
6     // 获取欧拉角（度）
7     float roll = ahrs.get_roll_deg();
8     float pitch = ahrs.get_pitch_deg();
9     float yaw = ahrs.get_yaw_deg();
10
11     hal.console->printf("Attitude: R=%.1f°, P=%.1f°, Y=%.1f°\n",
12                         roll, pitch, yaw);
13
14     // 获取旋转矩阵
15     Matrix3f rotation = ahrs.get_rotation_body_to_ned();
16
17     // 机体到地球坐标转换
18     Vector3f body_accel = {1.0f, 0.0f, 0.0f}; // 机体前方1g
19     Vector3f earth_accel = rotation * body_accel;
20
21     // 获取四元数
22     Quaternion quat;
23     ahrs.get_quat_body_to_ned(quat);
24
25     // ===== 2. 获取速度和位置 =====
26
27     Vector3f velocity;
28     if (ahrs.get_velocity_NED(velocity)) {
```

```

29     float speed = velocity.length();
30     float climb_rate = -velocity.z; // NED坐标系中Z向下
31
32     hal.console->printf("Speed: %.1f m/s, Climb: %.2f m/s\n",
33                         speed, climb_rate);
34 }
35
36 Location loc;
37 if (ahrs.get_location(loc)) {
38     hal.console->printf("Position: %.7f, %.7f, %.1fm\n",
39                         loc.lat / 1.0e7, loc.lng / 1.0e7, loc.alt /
100.0f);
40 }
41
42 // ===== 3. 获取风估计 =====
43
44 Vector3f wind = ahrs.wind_estimate();
45 if (wind.length() > 0.5f) { // 风速大于0.5 m/s
46     hal.console->printf("Wind: %.1f m/s from %.0f°\n",
47                         wind.length(), degrees(atan2f(wind.y, wind.x)));
48 }
49 }
50
51 // ===== 4. 坐标转换 =====
52
53 void coordinate_transform_example() {
54     AP_AHRS &ahrs = AP::ahrs();
55
56     // 机体坐标 → 地球坐标 (NED)
57     Vector3f body_vec = {1.0f, 0.0f, 0.0f}; // 机头方向
58     Vector3f earth_vec = ahrs.body_to_earth(body_vec);
59
60     // 地球坐标 → 机体坐标
61     Vector3f ned_vec = {1.0f, 0.0f, 0.0f}; // 北方
62     Vector3f body_result = ahrs.earth_to_body(ned_vec);
63 }

```

5.1.4 AHRS后端实现

```

1  enum class EKFTYPE : uint8_t {
2      DCM = 0,           // 方向余弦矩阵（简单、可靠）
3      TWO = 2,           // EKF2（扩展卡尔曼滤波v2，旧版本）
4      THREE = 3,        // EKF3（扩展卡尔曼滤波v3，**推荐**）
5      SIM = 10,          // SITL仿真专用
6      EXTERNAL = 11,     // 外部姿态估计器（如ROS）
7  };
8
9  /**
10   * 选择AHRS后端的考虑因素：
11   *
12   * DCM:
13   * + 简单可靠，计算量小
14   * + 无GPS也可工作
15   * - 精度不如EKF
16   * - 长距离飞行会有漂移

```

```

17  *
18  * EKF3:
19  * + 融合多传感器，精度高
20  * + 自动检测和拒绝异常数据
21  * + 支持多种传感器源
22  * - 计算量大
23  * - 需要GPS（室外）或光流/视觉（室内）
24  */

```

5.2 EKF3 - 扩展卡尔曼滤波

文件位置: `libraries/AP_NavEKF3/`

EKF3是ArduPilot**推荐使用的**导航滤波器，使用贝叶斯估计融合多传感器数据。

5.2.1 卡尔曼滤波原理

```

1  卡尔曼滤波核心思想：
2  基于【预测】（基于模型）+【更新】（基于测量）来估计系统状态
3
4  预测阶段（Predict）：
5       $\hat{x}_{n-1} = F \times x_{n-1} + B \times u_{n-1}$            // 状态预测
6       $P_{n-1} = F \times P_{n-1} \times F^T + Q$            // 协方差预测
7
8  更新阶段（Update）：
9       $K = P_{n-1} \times H^T \times (H \times P_{n-1} \times H^T + R)^{-1}$  // 卡尔曼增益
10      $\hat{x}_n = \hat{x}_{n-1} + K \times (z - H \times \hat{x}_{n-1})$        // 状态更新
11      $P_n = (I - K \times H) \times P_{n-1}$                    // 协方差更新
12
13  其中：
14  -  $\hat{x}$ ：状态估计
15  -  $P$ ：估计协方差（不确定性）
16  -  $F$ ：状态转移矩阵
17  -  $H$ ：观测矩阵
18  -  $Q$ ：过程噪声
19  -  $R$ ：测量噪声
20  -  $K$ ：卡尔曼增益
21  -  $z$ ：测量值

```

5.2.2 EKF3状态向量

```

1  /**
2   * EKF3估计的状态量：
3   */
4  struct EKFState {
5      // ===== 位置 (3) =====
6      float north;           // 北向位置 (m)
7      float east;            // 东向位置 (m)
8      float down;            // 高度 (m, 正值向下)
9
10     // ===== 速度 (3) =====
11     float vel_north;        // 北向速度 (m/s)
12     float vel_east;         // 东向速度 (m/s)
13     float vel_down;         // 垂直速度 (m/s)

```

```

14
15 // ===== 姿态 (4) =====
16 // 使用四元数表示 (4个变量, 但只有3个自由度)
17 Quaternion attitude; // 姿态四元数
18
19 // ===== 传感器偏差 (6) =====
20 Vector3f gyro_bias; // 陀螺仪零偏 (rad/s)
21 Vector3f accel_bias; // 加速度计零偏 (m/s²)
22
23 // ===== 磁场 (3, 可选) =====
24 Vector3f mag_bias; // 磁力计零偏
25
26 // ===== 风速 (2, 可选) =====
27 float wind_north; // 北向风速 (m/s)
28 float wind_east; // 东向风速 (m/s)
29 };
30
31 /**
32  * 总计: 最多21个状态变量
33  * 状态向量维度影响计算量
34  */

```

5.2.3 EKF3传感器融合

```

1 /**
2  * EKF3传感器融合流程:
3  */
4
5 // 预测步骤 (高频, 400Hz):
6 void ekf_predict(float gyro_rad[], float accel_ms2[], float dt) {
7     // 1. 使用陀螺仪积分更新姿态
8     // 2. 使用加速度计 (去除重力后) 更新速度
9     // 3. 积分速度更新位置
10    // 4. 估计传感器偏差
11 }
12
13 // 更新步骤 (低频, 各传感器频率不同):
14
15 // GPS位置更新 (5-20Hz):
16 void ekf_update_gps(float lat, float lon, float alt, float timestamp) {
17     // 将GPS位置与EKF预测位置比较
18     // 计算创新 (残差)
19     // 根据卡尔曼增益更新状态
20 }
21
22 // GPS速度更新 (5-20Hz):
23 void ekf_update_gps_vel(float v_north, float v_east, float v_down) {
24     // 融合GPS速度, 提高速度估计精度
25 }
26
27 // 气压计高度更新 (10-50Hz):
28 void ekf_update_baro(float altitude) {
29     // 融合气压高度
30     // 注意: 气压高度有漂移, 需要与GPS融合
31 }

```

```

32
33 // 磁力计航向更新（5-20Hz）：
34 void ekf_update_mag(float mag_x, float mag_y, float mag_z) {
35     // 融合磁力计航向
36     // 自动检测和校正磁场干扰
37 }
38
39 // 光流/视觉更新（高速）：
40 void ekf_update_optical_flow(float flow_x, float flow_y) {
41     // 在GPS信号弱时使用
42     // 需要已知高度
43 }

```

5.2.4 EKF3诊断

```

1 // ===== 1. 检查滤波器状态 =====
2
3 void check_ekf_status() {
4     AP_AHRS &ahrs = AP::ahrs();
5     nav_filter_status status;
6
7     ahrs.get_filter_status(status);
8
9     if (status.flags.attitude) {
10         // 姿态估计有效
11     }
12     if (status.flags.horiz_pos) {
13         // 水平位置估计有效
14     }
15     if (status.flags.vert_pos) {
16         // 垂直位置估计有效
17     }
18     if (status.flags.const_pos_mode) {
19         // 处于恒定位置模式（无GPS）
20     }
21 }
22
23 // ===== 2. 获取创新（残差） =====
24
25 void check_innovations() {
26     Vector3f vel_innov, pos_innov, mag_innov;
27     float tas_innov, yaw_innov;
28
29     AP::ahrs().get_innovations(vel_innov, pos_innov, mag_innov,
30                               tas_innov, yaw_innov);
31
32     // 创新值应该接近0
33     // 如果创新值过大，说明传感器有问题或EKF有问题
34     if (vel_innov.length() > 1.0f) {
35         // 速度创新异常
36     }
37 }
38
39 // ===== 3. 获取方差（不确定性） =====
40

```

```
41 void check_variances() {
42     float vel_var, pos_var, hgt_var;
43     vector3f mag_var;
44     float tas_var;
45
46     AP::ahrs().get_variances(vel_var, pos_var, hgt_var, mag_var, tas_var);
47
48     // 方差值越小，估计越可靠
49     // 方差 > 1.0 表示估计不可靠
50     if (pos_var > 1.0f) {
51         // 位置估计不可靠
52     }
53 }
```

5.3 DCM - 方向余弦矩阵

文件位置: `libraries/AP_AHRS/AP_AHRS_DCM.cpp`

DCM是最基础的姿态估计算法，原理简单、计算量小。

5.3.1 DCM原理

```
1 DCM表示从机体坐标系到地球坐标系的旋转矩阵：
2
3 DCM = [ X_body  →  NED ]
4       [ Y_body  →  NED ]
5       [ Z_body  →  NED ]
6
7 DCM矩阵示例（水平飞行，机头朝北）：
8       [ 1    0    0 ]    // x轴（机头）指向北
9       [ 0    1    0 ]    // Y轴（右侧）指向东
10      [ 0    0   -1 ]    // Z轴（向下）
11
12 更新公式：
13 DCM_new = DCM_old + DCM_old × (ω × dt)
14
15 其中ω是角速度向量，dt是时间步长
16
17 从DCM提取欧拉角：
18 roll  = atan2(DCM[1][2], DCM[2][2])
19 pitch = -asin(DCM[0][2])
20 yaw   = atan2(DCM[0][1], DCM[0][0])
```

5.3.2 DCM vs EKF

特性	DCM	EKF3
计算量	小	大
内存占用	小	大
精度	一般	高
传感器融合	仅陀螺仪+加速度计	多传感器融合

特性	DCM	EKF3
异常检测	无	有
适用场景	简单飞行、室内	室外长距离、精准作业

6. 控制系统

6.1 AC_AttitudeControl - 姿态控制

文件位置: `libraries/AC_AttitudeControl/`

姿态控制器根据**目标姿态**计算**电机输出**。

6.1.1 控制层级

1	飞控控制层级:
2	
3	1. 位置控制器 (Position Controller)
4	输入: 目标位置 (x, y, z)
5	输出: 目标姿态角 + 目标油门
6	用途: 自动飞行、悬停
7	-----
8	2. 姿态控制器 (Attitude Controller)
9	输入: 目标姿态角 (roll, pitch, yaw)
10	输出: 目标角速度 (roll_rate, pitch_rate, yaw_rate)
11	算法: P控制器 + 前馈
12	-----
13	3. 角速度控制器 (Rate Controller)
14	输入: 目标角速度
15	输出: 各电机PWM差值
16	算法: PID控制器
17	-----
18	4. 电机混控器 (Motor Mixer)
19	输入: Roll/Pitch/Yaw/Throttle控制量
20	输出: 各电机PWM值
21	-----

6.1.2 多旋翼姿态控制类

1	<code>class AC_AttitudeControl_Multi : public AC_AttitudeControl {</code>
2	<code>public:</code>
3	<code>/**</code>
4	<code> * 设置目标姿态 (四元数)</code>
5	<code> * @param target_attitude 目标四元数</code>
6	<code> */</code>
7	<code>void set_attitude_target_quaternion(const Quaternion& target_attitude);</code>
8	
9	<code>/**</code>
10	<code> * 设置目标姿态 (欧拉角, 度)</code>
11	<code> * @param roll_deg 横滚角目标</code>
12	<code> * @param pitch_deg 俯仰角目标</code>
13	<code> * @param yaw_deg 偏航角目标</code>


```

14     */
15     void set_attitude_target_euler(float roll_deg, float pitch_deg, float
yaw_deg);
16
17     /**
18     * 设置目标角速度（度/秒）
19     * 这是rate控制器的输入
20     */
21     void set_angvel_target(const Vector3f& angvel_rads);
22
23     /**
24     * 执行角速度控制（每帧调用）
25     * 从set_angvel_target()获取目标，计算电机输出
26     */
27     void rate_controller_run();
28
29     /**
30     * 设置油门输出
31     * @param throttle_in 油门输入（0-1）
32     * @param apply_angle_correction 是否应用角度校正
33     */
34     void set_throttle_out(float throttle_in, bool apply_angle_correction);
35
36     /**
37     * 设置悬停油门（平衡点）
38     */
39     void set_hover_throttle(float throttle) { _hover_throttle = throttle; }
40 };

```

6.1.3 使用示例

```

1 // ===== 1. 设置目标姿态 =====
2
3 void attitude_control_example() {
4     AC_AttitudeControl *attitude_control = copter.attitude_control;
5
6     // 目标：水平悬停
7     attitude_control->set_attitude_target_euler(0.0f, 0.0f, 0.0f);
8
9     // 目标：右倾10度
10    attitude_control->set_attitude_target_euler(10.0f, 0.0f, 0.0f);
11
12    // 目标：前倾5度，右转30度
13    attitude_control->set_attitude_target_euler(0.0f, 5.0f, 30.0f);
14 }
15
16 // ===== 2. 角速度控制 =====
17
18 void rate_control_example() {
19     AC_AttitudeControl *attitude_control = copter.attitude_control;
20
21     // 目标角速度（度/秒）
22     Vector3f target_rates;
23     target_rates.x = 100.0f; // 向右滚转
24     target_rates.y = 0.0f;

```

```
25     target_rates.z = 0.0f;
26
27     // 设置目标角速度
28     attitude_control->set_angvel_target(target_rates);
29
30     // 执行控制（在fast_loop中每帧调用）
31     attitude_control->rate_controller_run();
32 }
33
34 // ===== 3. 设置油门 =====
35
36 void throttle_control_example() {
37     AC_AttitudeControl *attitude_control = copter.attitude_control;
38
39     // 设置悬停油门（50%油门平衡飞机重量）
40     attitude_control->set_hover_throttle(500); // PWM值
41
42     // 直接设置油门
43     attitude_control->set_throttle_out(500, true);
44 }
```

6.2 PID控制原理详解

1 PID控制器结构图:

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20 各参数作用:

21

22

23

24

25

26

27

28

29

30

31

32

The diagram illustrates the PID control loop. It starts with a '目标值' (Setpoint) and a '测量值' (Measured value) entering a 'P' (Proportional) block. The 'P' block outputs '误差' (Error), which is then fed into a '求和' (Summing junction) block. The '求和' block also receives feedback from the '测量值' and outputs to an 'I' (Integral) block. The 'I' block outputs to a 'D' (Derivative) block. The 'D' block also receives feedback from the '测量值' and outputs 'D输出' (Derivative output). The '求和' block also receives 'P输出' (Proportional output) and 'D输出' (Derivative output) and outputs to the 'I' block. The 'I' block outputs to the 'D' block. The final output is '输出到执行器' (Output to actuator).

P (比例)	Proportional
作用:	误差越大, 输出越大
公式:	$output = K_p \times error$
优点:	响应快
缺点:	可能振荡, 无法消除稳态误差
I (积分)	Integral
作用:	累积误差, 消除稳态误差
公式:	$output = K_i \times \int error dt$

33		优点：消除稳态误差	
34		缺点：积分饱和、超调	
35			
36		D (微分) Derivative	
37			
38		作用：预测误差趋势，增加阻尼	
39		公式： $\text{output} = K_d \times d(\text{error})/dt$	
40		优点：减少振荡、改善稳定性	
41		缺点：对噪声敏感	
42			
43			
44		调参口诀：	
45			
46		1. 先调P，找到系统振荡的临界点	
47		2. 再调D，增加阻尼消除振荡	
48		3. 最后调I，消除稳态误差	
49			
50		P太大 → 系统振荡	
51		I太小 → 稳态误差（位置不准）	
52		D太大 → 噪声敏感、抖动	
53			

6.3 电机混控

1		多旋翼电机混控原理：
2		
3		四旋翼X模式：
4		前
5		↑
6		M1
7		
8		M2 ← → M4
9		
10		M3
11		↓
12		后
13		
14		电机编号（逆时针）：
15		M1：前右 (CW)
16		M2：后右 (CCW)
17		M3：后左 (CCW)
18		M4：前左 (CW)
19		
20		混控公式：
21		$\text{motor1} = \text{throttle} + \text{roll} + \text{pitch} + \text{yaw}$
22		$\text{motor2} = \text{throttle} - \text{roll} + \text{pitch} - \text{yaw}$
23		$\text{motor3} = \text{throttle} - \text{roll} - \text{pitch} + \text{yaw}$
24		$\text{motor4} = \text{throttle} + \text{roll} - \text{pitch} - \text{yaw}$
25		
26		电机转向：
27		CW（顺时针）产生反扭矩
28		CCW（逆时针）产生反扭矩
29		
30		姿态控制时的电机变化：
31		- 右滚 (Roll+)：M1、M4加速，M2、M3减速 → 产生向右的力矩

32	- 前俯 (Pitch+): M1、M4减速, M2、M3加速 → 产生向前的力矩
33	- 右转 (Yaw+): M1、M2加速, M3、M4减速 → 产生顺时针扭矩

6.4 飞行模式

ArduPilot支持多种飞行模式：

模式	类型	说明	适用场景
Stabilize	手动	手动控制姿态，油门中立悬停	新手练习
Acro	手动	手动控制角速度，油门控制高度	特技飞行
AltHold	半自主	高度自动保持，手动水平控制	航拍
Loiter	自主	位置自动保持（GPS）	悬停拍摄
RTL	自动	自动返回起航点	紧急返航
Auto	自动	按任务航点飞行	自主作业
Guided	自动	地面站引导飞行	地面站控制
Circle	自动	绕圈飞行	环绕拍摄
PosHold	半自主	简化版位置保持	简单悬停
Brake	自动	急停悬停	快速停止
FollowMe	自动	跟随移动目标	跟随拍摄

6.5 控制算法数学原理与代码实现

本节深入讲解飞控核心算法的数学物理模型、公式推导及代码实现。

6.5.1 PID控制器 - 数学原理与离散实现

6.5.1.1 连续PID控制器公式

PID控制器是飞控中最常用的控制算法，其连续形式的标准公式为：

$$u(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau) d\tau + K_d \cdot \frac{de(t)}{dt}$$

其中：

- $u(t)$: 控制器输出
- $e(t) = r(t) - y(t)$: 目标值与测量值的误差
- K_p : 比例增益
- K_i : 积分增益
- K_d : 微分增益

各分量物理意义：

分量	数学表达式	物理意义	对系统的影响
比例项 P	$K_p \cdot e$	误差的瞬时放大	响应速度，与误差成正比
积分项 I	$K_i \cdot \int e dt$	误差的累积	消除稳态误差，但可能导致超调
微分项 D	$K_d \cdot de/dt$	误差变化率	增加阻尼，抑制振荡

6.5.1.2 离散PID控制器（数字实现）

由于飞控使用数字处理器，必须将连续公式离散化。采用后向欧拉法进行离散化：

离散化推导：

积分项离散化：

$$\int_0^t e(\tau) d\tau \approx \sum_{n=0}^k e[n] \cdot \Delta t$$

微分项离散化：

$$\frac{de(t)}{dt} \approx \frac{e[k] - e[k-1]}{\Delta t}$$

离散PID公式：

$$u[k] = K_p \cdot e[k] + K_i \cdot \sum_{n=0}^k e[n] \cdot \Delta t + K_d \cdot \frac{e[k] - e[k-1]}{\Delta t}$$

其中：

- k : 当前采样时刻
- Δt : 采样周期（控制周期）

6.5.1.3 ArduPilot PID实现源码解析

文件位置: `libraries/PID/PID.cpp`

```
1  /**
2   * PID控制器 - 离散实现
3   *
4   * 数学公式:
5   * u[k] = kp*e[k] + ki*Σ(e[n]) + kd*de/dt
6   *
7   * 特点:
8   * 1. 使用低通滤波平滑微分项（避免噪声放大）
9   * 2. 积分限幅防止积分饱和
10  * 3. 自动微分计算，处理测量值跳变
11  */
12
13 #include "PID.h"
14 #include <AP_HAL/AP_HAL.h>
15
16 // PID参数默认值
17 const AP_Param::GroupInfo PID::var_info[] = {
18     AP_PARAMINFO(_kp),
19     AP_PARAMINFO(_ki),
20     AP_PARAMINFO(_kd),
21     AP_PARAMINFO(_kf),
22     AP_PARAMINFO(_imax),
23     AP_PARAMINFO(_fCut),
24     AP_PARAMEND
25 };
```

```

26
27 /**
28  * 计算PID输出
29  *
30  * @param error    误差值（目标值 - 测量值）
31  * @param scaler    缩放因子（用于多旋翼油门混控）
32  * @return          PID输出值
33  *
34  * 数学流程：
35  * 1. 计算P项：P = Kp * error
36  * 2. 计算D项：D = Kd * d(error)/dt（低通滤波后）
37  * 3. 计算I项：I = Ki * Σ(error)（带限幅）
38  * 4. 输出：u = P + I + D
39  */
40 float PID::get_pid(float error, float scaler)
41 {
42     // ===== 1. 时间计算 =====
43     uint32_t tnow = AP_HAL::millis();           // 当前系统时间（毫
秒）
44     uint32_t dt = tnow - _last_t;              // 采样间隔
45     float output = 0;                          // 初始化输出
46     float delta_time;                          // 采样间隔（秒）
47
48     /**
49      * 首次运行或长时间中断后的处理：
50      * - dt > 1000ms（1秒）：认为系统刚启动，重置积分项
51      * - 避免积分项累积过大的初始误差
52      */
53     if (_last_t == 0 || dt > 1000) {
54         dt = 0;
55         reset_I();                             // 重置积分器
56     }
57     _last_t = tnow;                            // 更新上次时间
58     delta_time = (float)dt * 0.001f;           // 转换为秒
59
60     // ===== 2. 比例项（P） =====
61     // 公式：P = Kp * error
62     _pid_info.P = error * _kp;                // 比例分量
63     output += _pid_info.P;                    // 加入总输出
64
65     // ===== 3. 微分项（D） - 带低通滤波 =====
66     /**
67      * 微分项的噪声问题：
68      * - 微分运算会放大高频噪声
69      * - 解决方案：使用一阶低通滤波平滑
70      *
71      * 一阶低通滤波公式：
72      *  $y[k] = \alpha * x[k] + (1-\alpha) * y[k-1]$ 
73      * 其中  $\alpha = \Delta t / (RC + \Delta t)$ 
74      * RC：时间常数（fCut为截止频率， $RC = 1/(2\pi fCut)$ ）
75      */
76     if ((fabsf(_kd) > 0) && (dt > 0)) {
77         float derivative;                    // 原始微分值
78         float RC;                          // RC时间常数
79
80         // 首次使用：初始化微分值

```

```

81     if (isnan(_last_derivative)) {
82         derivative = 0;
83         _last_derivative = 0;
84     } else {
85         // 原始微分计算（后向差分）
86         //  $de/dt \approx (e[k] - e[k-1]) / \Delta t$ 
87         derivative = (error - _last_error) / delta_time;
88     }
89
90     // 低通滤波平滑微分值
91     // 截止频率fCut越高，滤波效果越弱（保留更多高频信号）
92     RC = 1.0f / (2.0f * M_PI * _fCut);
93     /**
94      * 滤波系数推导：
95      *  $\alpha = \Delta t / (RC + \Delta t)$ 
96      * 当 $\Delta t \ll RC$ 时， $\alpha \approx \Delta t / RC$ （小步长平滑）
97      * 当 $\Delta t \gg RC$ 时， $\alpha \approx 1$ （几乎不滤波）
98      */
99     derivative = _last_derivative +
100         ((delta_time / (RC + delta_time)) *
101         (derivative - _last_derivative));
102
103     // 存储原始微分值用于下一次迭代
104     _last_error = error;
105     _last_derivative = derivative;
106
107     // 应用微分增益
108     _pid_info.D = derivative * _kd;
109     output += _pid_info.D;
110 }
111
112 // ===== 4. 积分项 (I) - 带限幅 =====
113 /**
114  * 积分项的作用：
115  * - 累积误差，消除稳态误差
116  * - 对于飞控：即使有持续的外力（如风），也能保持位置
117  *
118  * 积分饱和问题：
119  * - 积分器会不断累积，即使输出已被限制
120  * - 解决方法：限制积分项的幅度
121  */
122 if (fabsf(_ki) > 0) {
123     // 积分累加： $I = I + K_i * error * \Delta t$ 
124     _integrator += error * delta_time * _ki;
125
126     // 积分限幅
127     // 限制积分器输出不超过一定范围，防止积分饱和
128     float integrator_max = _imax / fabsf(_ki);
129     _integrator = constrain_float(_integrator, -integrator_max,
integrator_max);
130
131     _pid_info.I = _integrator;
132     output += _integrator;
133 }
134
135 // ===== 5. 应用缩放因子 =====

```

```

136  /**
137   * 缩放因子的作用：
138   * - 用于多旋翼的油门混控
139   * - 当飞机倾斜时，需要增加油门以维持升力
140   * - 缩放因子 =  $1/\cos(\text{roll}) * 1/\cos(\text{pitch})$ 
141   */
142  if (scaler != 1.0f) {
143      output *= scaler;
144  }
145
146  // 限制最终输出范围
147  output = constrain_float(output, -_imax, _imax);
148
149  // 记录本次信息用于调试
150  _pid_info.err = error;
151  _pid_info.desired = 0;
152  _pid_info.FF = 0;
153
154  return output;
155 }
156
157 /**
158  * 获取角速度PID输出
159  *
160  * 与get_pid的区别：
161  * 1. 自动微分：不使用误差的微分，而使用测量值的变化率
162  * 2. 避免测量噪声放大：微分作用于测量值而非误差
163  *
164  * 公式：
165  *  $D = K_d * d(\text{measurement})/dt$  (而非  $d(\text{error})/dt$ )
166  */
167 float PID::get_pid_angvel(float error, float measurement, float scaler)
168 {
169     uint32_t tnow = AP_HAL::millis();
170     uint32_t dt = tnow - _last_t_angvel;
171     float output = 0;
172     float delta_time;
173
174     // 时间处理（与get_pid相同）
175     if (_last_t_angvel == 0 || dt > 1000) {
176         dt = 0;
177         reset_I();
178     }
179     _last_t_angvel = tnow;
180     delta_time = (float)dt * 0.001f;
181
182     // P项和I项（与get_pid相同）
183     _pid_info.P = error * _kp;
184     output += _pid_info.P;
185
186     if (fabsf(_ki) > 0) {
187         _integrator += error * delta_time * _ki;
188         float integrator_max = _imax / fabsf(_ki);
189         _integrator = constrain_float(_integrator, -integrator_max,
190 integrator_max);
191         _pid_info.I = _integrator;

```



```

191     output += _integrator;
192 }
193
194 // D项：使用测量值微分（自动微分）
195 /**
196  * 自动微分 vs 手动微分：
197  * - 手动微分：d(error)/dt = (target - measurement)/dt
198  * - 自动微分：d(measurement)/dt
199  *
200  * 自动微分的优势：
201  * - 目标值变化时项不会产生，D冲击响应
202  * - 更平滑的控制效果
203  */
204 if ((fabsf(_kd) > 0) && (dt > 0)) {
205     float derivative;
206
207     if (isnan(_last_derivative_angvel)) {
208         derivative = 0;
209         _last_derivative_angvel = 0;
210     } else {
211         // 测量值微分（而非误差微分）
212         derivative = (measurement - _last_measurement) / delta_time;
213     }
214
215     float RC = 1.0f / (2.0f * M_PI * _fCut);
216     derivative = _last_derivative_angvel +
217                 ((delta_time / (RC + delta_time)) *
218                  (derivative - _last_derivative_angvel));
219
220     _last_measurement = measurement;
221     _last_derivative_angvel = derivative;
222
223     _pid_info.D = derivative * _kd;
224     output += _pid_info.D;
225 }
226
227 if (scaler != 1.0f) {
228     output *= scaler;
229 }
230
231 output = constrain_float(output, -_imax, _imax);
232
233 _pid_info.err = error;
234 _pid_info.desired = 0;
235
236 return output;
237 }
238
239 /**
240  * 重置积分器
241  *
242  * 使用场景：
243  * 1. 控制器切换模式
244  * 2. 检测到异常状态
245  * 3. 手动重置
246  */

```

```

247 void PID::reset_I()
248 {
249     _integrator = 0;
250     _last_derivative = NAN;      // 标记为未初始化
251     _last_derivative_angvel = NAN;
252 }

```

6.5.1.4 PID参数调谐指南

调参原则：

```

1 调参顺序：P → D → I
2
3
4 | 第一步：调节比例增益Kp |
5 |_____|
6 | 1. 设置 Ki = 0, Kd = 0 |
7 | 2. 逐渐增大Kp直到： |
8 |   - 系统开始振荡（临界增益） |
9 |   - 记录此时的Kp值 |
10 | 3. 设置工作Kp = 0.5 ~ 0.7 × 临界增益 |
11 |_____|
12 | 判断标准： |
13 | - Kp太小：响应慢，误差消除慢 |
14 | - Kp合适：快速响应，轻微超调 |
15 | - Kp太大：持续振荡，无法稳定 |
16 |_____|
17 | 第二步：调节微分增益Kd |
18 |_____|
19 | 1. 在合适Kp基础上，逐渐增大Kd |
20 | 2. 目标：消除振荡，增加阻尼 |
21 |_____|
22 | 判断标准： |
23 | - Kd太小：仍有振荡 |
24 | - Kd合适：振荡快速衰减 |
25 | - Kd太大：响应迟缓，抖动 |
26 |_____|
27 | 第三步：调节积分增益Ki |
28 |_____|
29 | 1. 逐渐增大Ki直到稳态误差可接受 |
30 | 2. 观察：是否出现超调增大或积分饱和现象 |
31 |_____|
32 | 判断标准： |
33 | - Ki太小：稳态误差无法消除 |
34 | - Ki合适：稳态误差快速消除 |
35 | - Ki太大：超调增大，可能振荡 |
36 |_____|

```

多旋翼典型PID参数范围：

```

1 // 角速度环PID参数（AC_AttitudeControl中）
2 // Roll/Pitch角速度环
3 // Kp ≈ 0.15 ~ 0.25 （单位：rad/s per rad）
4 // Ki ≈ 0.0 ~ 0.02
5 // Kd ≈ 0.0 ~ 0.5 （实际使用自动微分）

```

```

6
7 #define ATC_RAT_RLL_P          0.15f // Roll角速度P
8 #define ATC_RAT_RLL_I          0.0f  // Roll角速度I
9 #define ATC_RAT_RLL_D          0.0f  // Roll角速度D（使用自动微分）
10 #define ATC_RAT_RLL_IMAX      0.0f  // Roll角速度积分限幅
11
12 #define ATC_RAT_PIT_P          0.15f // Pitch角速度P
13 #define ATC_RAT_PIT_I          0.0f  // Pitch角速度I
14 #define ATC_RAT_PIT_D          0.0f
15 #define ATC_RAT_PIT_IMAX      0.0f
16
17 // Yaw角速度环（需要更高增益，因为扭矩常数较小）
18 #define ATC_RAT_YAW_P          0.20f // Yaw角速度P
19 #define ATC_RAT_YAW_I          0.02f // Yaw角速度I（需要消除偏航漂移）
20 #define ATC_RAT_YAW_D          0.0f
21 #define ATC_RAT_YAW_IMAX      0.0f

```

6.5.2 四元数 - 3D旋转的数学原理

6.5.2.1 为什么使用四元数

欧拉角的万向节锁问题：

```

1 欧拉角旋转顺序：Z → Y → X
2
3 当 Pitch = ±90° 时：
4 - Roll轴和Yaw轴绕同一轴旋转
5 - 失去一个自由度
6 - 无法表示某些旋转
7
8 四元数优势：
9 1. 任意轴旋转，无万向节锁
10 2. 球面插值（slerp）平滑
11 3. 计算效率高（4个元素 vs 9个矩阵元素）
12 4. 数值稳定性好

```

6.5.2.2 四元数数学定义

四元数定义：

$$q = q_0 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k}$$

其中虚数单位满足：

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$$

$$\mathbf{ij} = \mathbf{k}, \quad \mathbf{ji} = -\mathbf{k}$$

$$\mathbf{jk} = \mathbf{i}, \quad \mathbf{kj} = -\mathbf{i}$$

$$\mathbf{ki} = \mathbf{j}, \quad \mathbf{ik} = -\mathbf{j}$$

四元数表示旋转：

$$q = \begin{bmatrix} q_0 \\ \vec{q} \end{bmatrix} = \begin{bmatrix} \cos(\theta/2) \\ \vec{n} \sin(\theta/2) \end{bmatrix}$$

其中：

- θ ：绕轴旋转角度
- $\vec{n}$ ：单位旋转轴向量

模长约束 (单位四元数) :

$$|q| = q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1$$

6.5.2.3 四元数乘法 (旋转合成)

Hamilton积:

$$q_1 \otimes q_2 = \begin{bmatrix} q_{10}q_{20} - \vec{q}_1 \cdot \vec{q}_2 \\ q_{10}\vec{q}_2 + q_{20}\vec{q}_1 + \vec{q}_1 \times \vec{q}_2 \end{bmatrix}$$

旋转合成公式:

若 q_1 表示旋转A, q_2 表示旋转B, 则:

$$q_{total} = q_2 \otimes q_1$$

表示先应用A, 再应用B (注意顺序!)。

6.5.2.4 四元数旋转向量

向量旋转公式:

给定单位四元数 q 和向量 $\vec{v}$, 旋转后的向量:

$$v' = q \otimes \begin{bmatrix} 0 \\ \vec{v} \end{bmatrix} \otimes q^*$$

$$\text{其中共轭四元数 } q^* = \begin{bmatrix} q_0 \\ -\vec{q} \end{bmatrix}$$

展开公式:

$$\vec{v}' = \vec{v} + 2q_0(\vec{q} \times \vec{v}) + 2(\vec{q} \times (\vec{q} \times \vec{v}))$$

6.5.2.5 四元数与欧拉角转换

欧拉角转四元数 (Z-Y-X顺序) :

$$q_0 = \cos(\phi/2) \cos(\theta/2) \cos(\psi/2) + \sin(\phi/2) \sin(\theta/2) \sin(\psi/2)$$

$$q_1 = \sin(\phi/2) \cos(\theta/2) \cos(\psi/2) - \cos(\phi/2) \sin(\theta/2) \sin(\psi/2)$$

$$q_2 = \cos(\phi/2) \sin(\theta/2) \cos(\psi/2) + \sin(\phi/2) \cos(\theta/2) \sin(\psi/2)$$

$$q_3 = \cos(\phi/2) \cos(\theta/2) \sin(\psi/2) - \sin(\phi/2) \sin(\theta/2) \cos(\psi/2)$$

四元数转欧拉角 (Z-Y-X顺序) :

$$\phi = \arctan 2(2(q_0q_1 + q_2q_3), 1 - 2(q_1^2 + q_2^2))$$

$$\theta = \arcsin(2(q_0q_2 - q_3q_1))$$

$$\psi = \arctan 2(2(q_0q_3 + q_1q_2), 1 - 2(q_2^2 + q_3^2))$$

6.5.2.6 ArduPilot四元数代码实现

文件位置: `libraries/AP_Math/quaternion.h`

```
1 /**
2  * 四元数类 - 3D旋转的高效表示
3  *
4  * 数学基础:
5  * q = [w, x, y, z] = [cos(θ/2), n_x*sin(θ/2), n_y*sin(θ/2), n_z*sin(θ/2)]
6  * 其中 n = [n_x, n_y, n_z] 是单位旋转轴, θ 是旋转角度
```

```

7  */
8
9  #pragma once
10
11 #include <AP_Math/AP_Math.h>
12
13 class Matrix3f;
14
15 /**
16  * 四元数类定义
17  *
18  * 数据成员:
19  * q0: 实部 (scalar part)
20  * q1, q2, q3: 虚部 (vector part)
21  */
22 class Quaternion {
23 public:
24     float q0, q1, q2, q3;          // 四元数四个分量
25
26     /**
27      * 构造函数 - 初始化为单位四元数 (无旋转)
28      */
29     Quaternion() :
30         q0(1.0f), q1(0.0f), q2(0.0f), q3(0.0f) {}
31
32     /**
33      * 构造函数 - 直接指定四元数分量
34      */
35     Quaternion(float _q0, float _q1, float _q2, float _q3) :
36         q0(_q0), q1(_q1), q2(_q2), q3(_q3) {}
37
38     /**
39      * 从欧拉角构造四元数
40      *
41      * 数学公式: Z-Y-X旋转顺序
42      * 先绕Z轴旋转yaw, 再绕Y轴旋转pitch, 最后绕X轴旋转roll
43      *
44      * @param roll_deg 横滚角 (度)
45      * @param pitch_deg 俯仰角 (度)
46      * @param yaw_deg 偏航角 (度)
47      */
48     void from_euler(float roll_deg, float pitch_deg, float yaw_deg)
49     {
50         // 转换为弧度
51         float roll_rad = radians(roll_deg);
52         float pitch_rad = radians(pitch_deg);
53         float yaw_rad = radians(yaw_deg);
54
55         // 计算半角
56         float cr = cosf(roll_rad * 0.5f);
57         float cp = cosf(pitch_rad * 0.5f);
58         float cy = cosf(yaw_rad * 0.5f);
59
60         float sr = sinf(roll_rad * 0.5f);
61         float sp = sinf(pitch_rad * 0.5f);
62         float sy = sinf(yaw_rad * 0.5f);

```

```

63
64 // 四元数计算公式 (Z-Y-X顺序)
65 // q = q_yaw ⊗ q_pitch ⊗ q_roll
66 q0 = cr * cp * cy + sr * sp * sy;
67 q1 = sr * cp * cy - cr * sp * sy;
68 q2 = cr * sp * cy + sr * cp * sy;
69 q3 = cr * cp * sy - sr * sp * cy;
70 }
71
72 /**
73  * 将四元数转换为欧拉角
74  *
75  * 数学公式:
76  * roll = atan2(2(q0q1 + q2q3), 1 - 2(q12 + q22))
77  * pitch = asin(2(q0q2 - q3q1))
78  * yaw = atan2(2(q0q3 + q1q2), 1 - 2(q22 + q32))
79  *
80  * @return Vector3f(x=roll, y=pitch, z=yaw) 单位: 度
81  */
82 Vector3f to_euler() const
83 {
84     Vector3f euler;
85
86     // Roll (X轴旋转)
87     /**
88      * 推导:
89      * 从四元数到旋转矩阵的第三列:
90      * R = [r11 r12 r13]
91      *      [r21 r22 r23]
92      *      [r31 r32 r33]
93      * roll = atan2(r32, r33)
94      *        = atan2(2(q0q1 + q2q3), q02 + q32 - q12 - q22)
95      *        = atan2(2(q0q1 + q2q3), 1 - 2(q12 + q22))
96      */
97     euler.x = degrees(atan2f(2.0f * (q0 * q1 + q2 * q3),
98                             1.0f - 2.0f * (q1 * q1 + q2 * q2)));
99
100    // Pitch (Y轴旋转)
101    /**
102     * 推导:
103     * pitch = asin(r31)
104     *        = asin(2(q0q2 - q3q1))
105     */
106    euler.y = degrees(asinf(2.0f * (q0 * q2 - q3 * q1)));
107
108    // Yaw (Z轴旋转)
109    /**
110     * 推导:
111     * yaw = atan2(r21, r11)
112     *       = atan2(2(q0q3 + q1q2), q02 + q12 - q22 - q32)
113     *       = atan2(2(q0q3 + q1q2), 1 - 2(q22 + q32))
114     */
115    euler.z = degrees(atan2f(2.0f * (q0 * q3 + q1 * q2),
116                            1.0f - 2.0f * (q2 * q2 + q3 * q3)));
117
118    return euler;

```

```

119     }
120
121     /**
122      * 四元数乘法 - 旋转合成
123      *
124      * 数学公式:
125      *  $q_1 \otimes q_2 = [q_{1_0}q_{2_0} - q_1 \cdot q_2, q_{1_0}q_2 + q_{2_0}q_1 + q_1 \times q_2]$ 
126      *
127      * 使用场景:
128      * 1. 连续旋转合成
129      * 2. 角速度积分 ( $dq/dt = 0.5 * \omega \otimes q$ )
130      *
131      * @param v 右侧四元数
132      * @return 乘积四元数
133      */
134     Quaternion operator*(const Quaternion &v) const
135     {
136         Quaternion result;
137
138         // Hamilton积展开
139         result.q0 = q0 * v.q0 - q1 * v.q1 - q2 * v.q2 - q3 * v.q3;
140         result.q1 = q0 * v.q1 + q1 * v.q0 + q2 * v.q3 - q3 * v.q2;
141         result.q2 = q0 * v.q2 - q1 * v.q3 + q2 * v.q0 + q3 * v.q1;
142         result.q3 = q0 * v.q3 + q1 * v.q2 - q2 * v.q1 + q3 * v.q0;
143
144         return result;
145     }
146
147     /**
148      * 从轴角构造四元数
149      *
150      * 数学公式:
151      *  $q = [\cos(\theta/2), n_x \sin(\theta/2), n_y \sin(\theta/2), n_z \sin(\theta/2)]$ 
152      *
153      * @param angle_rad 旋转角度 (弧度)
154      * @param axis 旋转轴 (单位向量)
155      */
156     void from_axis_angle(float angle_rad, const Vector3f &axis)
157     {
158         float half_angle = angle_rad * 0.5f;
159         float s = sinf(half_angle);
160
161         q0 = cosf(half_angle);
162         q1 = axis.x * s;
163         q2 = axis.y * s;
164         q3 = axis.z * s;
165     }
166
167     /**
168      * 四元数归一化
169      *
170      * 数学公式:
171      *  $q_{normalized} = q / |q|$ 
172      *  $|q| = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2}$ 
173      *
174      * 重要性:

```

```

175     * - 浮点误差会导致四元数不再单位化
176     * - 非单位四元数表示错误的旋转角度
177     */
178 void normalize()
179 {
180     float norm = sqrtf(q0 * q0 + q1 * q1 + q2 * q2 + q3 * q3);
181     if (norm > 0) {
182         float inv_norm = 1.0f / norm;
183         q0 *= inv_norm;
184         q1 *= inv_norm;
185         q2 *= inv_norm;
186         q3 *= inv_norm;
187     }
188 }
189
190 /**
191  * 获取四元数的共轭
192  *
193  * 数学公式:
194  *  $q^* = [q_0, -q_1, -q_2, -q_3]$ 
195  *
196  * 用途:
197  * - 计算逆旋转:  $q^{-1} = q^* / |q|^2$ 
198  * - 对于单位四元数,  $q^{-1} = q^*$ 
199  */
200 Quaternion operator-(void) const
201 {
202     return Quaternion(q0, -q1, -q2, -q3);
203 }
204
205 /**
206  * 旋转向量
207  *
208  * 数学公式:
209  *  $v' = q \otimes [0, v] \otimes q^*$ 
210  *
211  * 展开:
212  *  $v' = v + 2q_0(q \times v) + 2(q \times (q \times v))$ 
213  *
214  * @param vec 输入向量 (body frame)
215  * @return 旋转后的向量 (NED frame或相反)
216  */
217 Vector3f rotate(const Vector3f &vec) const
218 {
219     // 四元数旋转向量的高效实现
220     // 避免临时四元数对象的创建
221
222     float q0q0 = q0 * q0;
223     float q1q1 = q1 * q1;
224     float q2q2 = q2 * q2;
225     float q3q3 = q3 * q3;
226
227     // 旋转矩阵元素 (从四元数构建)
228     float r00 = q0q0 + q1q1 - q2q2 - q3q3;
229     float r01 = 2.0f * (q1 * q2 - q0 * q3);
230     float r02 = 2.0f * (q1 * q3 + q0 * q2);

```



```

231     float r10 = 2.0f * (q1 * q2 + q0 * q3);
232     float r11 = q0q0 - q1q1 + q2q2 - q3q3;
233     float r12 = 2.0f * (q2 * q3 - q0 * q1);
234     float r20 = 2.0f * (q1 * q3 - q0 * q2);
235     float r21 = 2.0f * (q2 * q3 + q0 * q1);
236     float r22 = q0q0 - q1q1 - q2q2 + q3q3;
237
238     // 矩阵-向量乘法
239     Vector3f result;
240     result.x = r00 * vec.x + r01 * vec.y + r02 * vec.z;
241     result.y = r10 * vec.x + r11 * vec.y + r12 * vec.z;
242     result.z = r20 * vec.x + r21 * vec.y + r22 * vec.z;
243
244     return result;
245 }
246 };

```

6.5.3 旋转矩阵（方向余弦矩阵 DCM）

6.5.3.1 DCM数学定义

方向余弦矩阵是一个3×3正交矩阵，描述了两个坐标系之间的旋转关系：

$$R = [\vec{r}_x \quad \vec{r}_y \quad \vec{r}_z]$$

其中 $\vec{r}_x, \vec{r}_y, \vec{r}_z$ 是行向量（NED坐标系下的基向量）。

基本旋转矩阵：

绕X轴旋转（Roll）：

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}$$

绕Y轴旋转（Pitch）：

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

绕Z轴旋转（Yaw）：

$$R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

复合旋转（Z-Y-X顺序）：

$$R = R_x(\phi)R_y(\theta)R_z(\psi)$$

6.5.3.2 ArduPilot旋转矩阵代码

文件位置： `libraries/AP_Math/matrix3.h`

```

1  /**
2   * 3×3 旋转矩阵类
3   *
4   * 用于表示坐标系变换和向量旋转
5   *
6   * 数学基础：

```

```

7  * R = [r11 r12 r13] = [ex_x ey_x ez_x]
8  *      [r21 r22 r23]   [ex_y ey_y ez_y]
9  *      [r31 r32 r33]   [ex_z ey_z ez_z]
10 *
11 * 其中 ex, ey, ez 是机体坐标系基向量在NED坐标系的表示
12 */
13
14 #pragma once
15
16 #include <AP_Math/AP_Math.h>
17
18 class Quaternion;
19
20 class Matrix3f {
21 public:
22     float a.x, a.y, a.z;    // 第一行
23     float b.x, b.y, b.z;    // 第二行
24     float c.x, c.y, c.z;    // 第三行
25
26     /**
27      * 从欧拉角构造旋转矩阵
28      *
29      * 公式:  $R = R_x(\text{roll}) \times R_y(\text{pitch}) \times R_z(\text{yaw})$ 
30      *
31      * 完整展开:
32      * 
$$\begin{bmatrix} \cos\psi\cos\theta & -\sin\psi\cos\phi+\cos\psi\sin\theta\sin\phi & \sin\psi\sin\phi+\cos\psi\sin\theta\cos\phi \\ \sin\psi\cos\theta & \cos\psi\cos\phi+\sin\psi\sin\theta\sin\phi & -\cos\psi\sin\phi+\sin\psi\sin\theta\cos\phi \\ -\sin\theta & \cos\theta\sin\phi & \cos\theta\cos\phi \end{bmatrix}$$

33      *
34      * @param roll_rad 横滚角 (弧度)
35      * @param pitch_rad 俯仰角 (弧度)
36      * @param yaw_rad 偏航角 (弧度)
37      */
38     void from_euler(float roll_rad, float pitch_rad, float yaw_rad)
39     {
40         float cr = cosf(roll_rad);
41         float cp = cosf(pitch_rad);
42         float cy = cosf(yaw_rad);
43         float sr = sinf(roll_rad);
44         float sp = sinf(pitch_rad);
45         float sy = sinf(yaw_rad);
46
47         // 第一行: X轴方向余弦
48         a.x = cy * cp;
49         a.y = sy * cp;
50         a.z = -sp;
51
52         // 第二行: Y轴方向余弦
53         b.x = cy * sp * sr - sy * cr;
54         b.y = sy * sp * sr + cy * cr;
55         b.z = cp * sr;
56
57         // 第三行: Z轴方向余弦

```

```

60     c.x = cy * sp * cr + sy * sr;
61     c.y = sy * sp * cr - cy * sr;
62     c.z = cp * cr;
63 }
64
65 /**
66  * 将旋转矩阵转换为欧拉角
67  *
68  * 公式:
69  * roll = atan2(r32, r33)
70  * pitch = asin(-r31)
71  * yaw = atan2(r21, r11)
72  */
73 Vector3f to_euler() const
74 {
75     Vector3f euler;
76
77     // Pitch from -sin(r31)
78     // 数值稳定性: 使用asin而非直接取反三角函数
79     euler.y = asinf(-c.z);
80
81     // Roll from r32/r33
82     euler.x = atan2f(b.z, a.z);
83
84     // Yaw from r21/r11
85     euler.y = atan2f(b.x, a.x);
86
87     return euler;
88 }
89
90 /**
91  * 向量变换 (旋转)
92  *
93  * 将向量从一个坐标系变换到另一个坐标系
94  *
95  * 数学公式:
96  *  $v_{ned} = R \times v_{body}$ 
97  *
98  * 或反向变换:
99  *  $v_{body} = R^T \times v_{ned}$ 
100  *
101  * @param vec 输入向量
102  * @return 变换后的向量
103  */
104 Vector3f operator*(const Vector3f &vec) const
105 {
106     Vector3f result;
107
108     // 矩阵乘法:  $R \times v$ 
109     result.x = a.x * vec.x + a.y * vec.y + a.z * vec.z;
110     result.y = b.x * vec.x + b.y * vec.y + b.z * vec.z;
111     result.z = c.x * vec.x + c.y * vec.y + c.z * vec.z;
112
113     return result;
114 }
115

```

```

116  /**
117   * 矩阵乘法:  $R1 \times R2$ 
118   *
119   * 复合旋转:
120   *  $R\_total = R1 \times R2$ 
121   * 表示先应用R2, 再应用R1
122   */
123 Matrix3f operator*(const Matrix3f &m) const
124 {
125     Matrix3f result;
126
127     // 行x列乘法
128     result.a.x = a.x * m.a.x + a.y * m.b.x + a.z * m.c.x;
129     result.a.y = a.x * m.a.y + a.y * m.b.y + a.z * m.c.y;
130     result.a.z = a.x * m.a.z + a.y * m.b.z + a.z * m.c.z;
131
132     result.b.x = b.x * m.a.x + b.y * m.b.x + b.z * m.c.x;
133     result.b.y = b.x * m.a.y + b.y * m.b.y + b.z * m.c.y;
134     result.b.z = b.x * m.a.z + b.y * m.b.z + b.z * m.c.z;
135
136     result.c.x = c.x * m.a.x + c.y * m.b.x + c.z * m.c.x;
137     result.c.y = c.x * m.a.y + c.y * m.b.y + c.z * m.c.y;
138     result.c.z = c.x * m.a.z + c.y * m.b.z + c.z * m.c.z;
139
140     return result;
141 }
142
143 /**
144   * 矩阵转置
145   *
146   * 数学公式:
147   *  $R^T = R^{-1}$  (旋转矩阵是正交矩阵)
148   *
149   * 用于反向变换
150   */
151 Matrix3f transposed() const
152 {
153     Matrix3f result;
154
155     // 转置: 行列互换
156     result.a.x = a.x; result.a.y = b.x; result.a.z = c.x;
157     result.b.x = a.y; result.b.y = b.y; result.b.z = c.y;
158     result.c.x = a.z; result.c.y = b.z; result.c.z = c.z;
159
160     return result;
161 }
162
163 /**
164   * 矩阵归一化
165   *
166   * 浮点误差会导致矩阵列向量不再正交
167   * 需要定期归一化以保持数值精度
168   *
169   * Gram-Schmidt正交化:
170   *  $r\_x' = r\_x$ 
171   *  $r\_y' = r\_y - (r\_x' \cdot r\_y)r\_x'$ 

```

```

172     * r_z' = r_x' × r_y'
173     */
174 void normalize()
175 {
176     Vector3f x(a.x, a.y, a.z);
177     Vector3f y(b.x, b.y, b.z);
178     Vector3f z(c.x, c.y, c.z);
179
180     // 第一列归一化
181     float x_norm = x.length();
182     if (x_norm > 0) {
183         x /= x_norm;
184     }
185
186     // 第二列正交化（减去投影）
187     float dot = x * y;
188     y = y - x * dot;
189
190     // 第二列归一化
191     float y_norm = y.length();
192     if (y_norm > 0) {
193         y /= y_norm;
194     }
195
196     // 第三列叉乘（保证右手坐标系）
197     z = x % y;
198
199     // 写回矩阵
200     a.x = x.x; a.y = x.y; a.z = x.z;
201     b.x = y.x; b.y = y.y; b.z = y.z;
202     c.x = z.x; c.y = z.y; c.z = z.z;
203 }
204 };

```

6.5.4 DCM姿态估计算法

6.5.4.1 DCM算法原理

DCM（方向余弦矩阵）算法通过融合陀螺仪和加速度计/磁力计数据来估计飞行器姿态。

核心方程：

$$\dot{R} = R \times \Omega$$

其中：

- R ：方向余弦矩阵
- Ω ：角速度矩阵（反对称矩阵）

角速度矩阵：

$$\Omega = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

两欧更新:

$$\dot{R}_{k+1} = \dot{R}_k + \dot{R}_k \times \Delta t$$

$$R_{k+1} = R_k \times (I + \Omega \times \Delta t)$$

6.5.4.2 DCM漂移校正

陀螺仪积分会产生漂移，需要使用加速度计（检测重力）和磁力计（检测北向）进行校正。

加速度计校正:

```
1  /**
2   * 误差计算：期望重力向量与测量的夹角
3   *
4   * 数学公式：
5   * error = desired_g × measured_g
6   *
7   * 其中：
8   * - desired_g = [0, 0, 1] (NED坐标系中重力向下)
9   * - measured_g = R × body_accel (变换到NED坐标系)
10  */
11 Vector3f error = desired_g % measured_g_normed;
12
13 /**
14  * 误差修正：
15  * - 将误差向量投影到各轴
16  * - 漂移校正仅影响绕水平轴的旋转
17  * - yaw轴漂移需要磁力计校正
18  */
19
20 // 忽略z轴误差（加速度计无法检测yaw漂移）
21 error.z = 0;
22
23 // 误差积分
24 drift_correction += error * KI * dt;
25
26 // 应用校正到陀螺仪
27 omega += drift_correction;
```

6.5.4.3 ArduPilot DCM实现源码

文件位置: `libraries/AP_AHRS/AP_AHRS_DCM.cpp`

```
1  /**
2   * DCM姿态估计算法实现
3   *
4   * 算法流程：
5   * 1. 陀螺仪积分更新DCM矩阵
6   * 2. 矩阵正交化 (Gram-Schmidt)
7   * 3. 误差校正 (加速度计 + 磁力计)
8   * 4. 输出姿态估计 (欧拉角/四元数/DCM)
9   */
10
11 #include <AP_AHRS/AP_AHRS_DCM.h>
12 #include <AP_Math/AP_Math.h>
13
```

```

14 // 增益参数
15 const float kp = 1.0f;           // 比例校正增益
16 const float ki = 0.001f;        // 积分校正增益（用于长期漂移校正）
17 const float omega_mag_limit = 0.5f; // 角速度限制
18
19 // 全局变量
20 Matrix3f dcm_matrix;             // DCM矩阵
21 Vector3f gyro_offset;           // 陀螺仪零偏估计
22 Vector3f drift_correction;       // 漂移校正积分
23
24 /**
25  * DCM更新 - 每周周期调用
26  *
27  * @param gyro      陀螺仪测量值（弧度/秒）
28  * @param accel     加速度计测量值（m/s²）
29  * @param mag       磁力计测量值（可选）
30  * @param dt        时间步长（秒）
31  */
32 void AP_AHRS_DCM::update(float gyro[3], float accel[3], float mag[3], float
dt)
33 {
34     Vector3f gyro_vec(gyro[0], gyro[1], gyro[2]);
35     Vector3f accel_vec(accel[0], accel[1], accel[2]);
36
37     // ===== 1. 陀螺仪积分 =====
38     /**
39      * DCM矩阵更新公式:
40      *  $\frac{dR}{dt} = R \times \Omega$ 
41      *  $\Omega = \begin{bmatrix} 0 & -wz & wy \\ wz & 0 & -wx \\ -wy & wx & 0 \end{bmatrix}$ 
42      *
43      * 离散化（一阶近似）:
44      *  $R_{new} = R_{old} + dR \times dt$ 
45      */
46     float wx = gyro_vec.x;
47     float wy = gyro_vec.y;
48     float wz = gyro_vec.z;
49
50     // 构建反对称矩阵（略去）
51
52     // 矩阵更新
53     Matrix3f delta_rotation;
54     delta_rotation.a.x = 0;          delta_rotation.a.y = -wz * dt;
55     delta_rotation.a.z = wy * dt;
56     delta_rotation.b.x = wz * dt;   delta_rotation.b.y = 0;
57     delta_rotation.b.z = -wx * dt;
58     delta_rotation.c.x = -wy * dt;  delta_rotation.c.y = wx * dt;
59     delta_rotation.c.z = 0;
60
61     dcm_matrix = dcm_matrix * (Matrix3f::identity() + delta_rotation);
62
63     // ===== 2. 漂移校正 =====
64     /**
65      * 校正误差来源:
66      * - 加速度计: 检测重力方向, 校正roll/pitch

```

```

66     * - 磁力计：检测北向，校正yaw
67     *
68     * 误差计算方法：叉积误差法
69     * error = desired × measured
70     *
71     * 期望重力向量（NED坐标系）：[0, 0, 1]
72     * 期望磁力向量（NED坐标系）：[1, 0, 磁偏角]
73     */
74
75 // 加速度计校正
76 Vector3f accel_earth = dcm_matrix * accel_vec; // 变换到NED坐标系
77 Vector3f desired_g(0, 0, 1); // 期望重力方向
78
79 // 归一化测量值
80 float accel_mag = accel_earth.length();
81 if (accel_mag > 0) {
82     accel_earth /= accel_mag;
83
84     // 计算误差向量（叉积）
85     Vector3f error = desired_g % accel_earth;
86
87     /**
88      * 校正原理：
89      * - 误差方向 = 校正方向
90      * - 误差大小 = 角度误差
91      * - 应用校正到陀螺仪积分
92      */
93
94     // 忽略z轴（加速度计无法检测yaw）
95     error.z = 0;
96
97     // 积分校正（消除长期漂移）
98     drift_correction += error * Ki * dt;
99
100    // 应用校正到DCM
101    dcm_matrix.a += error * Kp;
102    dcm_matrix.b += error * Kp;
103    dcm_matrix.c += error * Kp;
104 }
105
106 // 磁力计校正（如果可用）
107 if (mag != nullptr) {
108     Vector3f mag_vec(mag[0], mag[1], mag[2]);
109
110     // 归一化磁力计
111     float mag_mag = mag_vec.length();
112     if (mag_mag > 0) {
113         mag_vec /= mag_mag;
114
115         // 去除pitch影响（磁力计水平投影）
116         // 变换到水平面
117         Vector3f mag_horizontal = dcm_matrix * mag_vec;
118         mag_horizontal.z = 0;
119         mag_horizontal.normalize();
120
121         // 期望北向

```



```

122         Vector3f desired_north(1, 0, 0);
123
124         // 计算yaw误差
125         float mag_error = desired_north % mag_horizontal;
126
127         // 应用yaw校正
128         dcm_matrix.a += mag_error * Kp;
129         dcm_matrix.b += mag_error * Kp;
130         dcm_matrix.c += mag_error * Kp;
131     }
132 }
133
134 // ===== 3. 矩阵正交化 =====
135 /**
136  * 由于浮点误差，DCM矩阵会逐渐失去正交性
137  * 需要定期正交化
138  *
139  * Gram-Schmidt正交化:
140  *  $x' = x$ 
141  *  $y' = y - (x' \cdot y)x'$ 
142  *  $z' = x' \times y'$ 
143  */
144 dcm_matrix.normalize();
145
146 // ===== 4. 输出更新 =====
147 // 更新欧拉角
148 update_euler_from_dcm();
149
150 // 更新四元数
151 update_quaternion_from_dcm();
152 }

```

6.5.5 互补滤波器

6.5.5.1 滤波器原理

互补滤波器利用不同传感器在不同频率段的特性进行融合：

```

1  /**
2   * 一阶互补滤波器
3   *
4   * 数学公式:
5   *  $y[k] = \alpha * x[k] + (1-\alpha) * y[k-1]$ 
6   *
7   * 频率响应:
8   * - 低频:  $x[k]$ 被衰减,  $y[k-1]$ 通过 → 通过低频信号
9   * - 高频:  $x[k]$ 通过,  $y[k-1]$ 被衰减 → 通过高频信号
10  *
11  * 截止频率:
12  *  $f_{cut} = \alpha / (2\pi * \Delta t)$ 
13  */
14
15 float complementary_filter(float measurement, float estimate, float alpha,
float dt)

```

```

16 {
17     return alpha * measurement + (1 - alpha) * estimate;
18 }

```

6.5.5.2 低通滤波器

一阶低通滤波器公式：

$$y[k] = \alpha \cdot x[k] + (1 - \alpha) \cdot y[k - 1]$$

其中：

$$\alpha = \frac{\Delta t}{RC + \Delta t}$$

RC ：时间常数，与截止频率关系为 $f_c = \frac{1}{2\pi RC}$

```

1  /**
2   * 一阶低通滤波器实现
3   *
4   * 用途：
5   * - 平滑加速度计信号
6   * - 滤除陀螺仪高频噪声
7   * - 平滑微分项
8   */
9  class LowPassFilter {
10 private:
11     float alpha;          // 滤波系数
12     float output;         // 上次输出
13     bool initialized;     // 初始化标志
14
15 public:
16     LowPassFilter() : output(0), initialized(false) {}
17
18     /**
19      * 设置截止频率
20      *
21      * @param fc    截止频率 (Hz)
22      * @param dt    采样周期 (秒)
23      */
24     void set_cutoff_frequency(float fc, float dt)
25     {
26         float RC = 1.0f / (2.0f * M_PI * fc);
27         alpha = dt / (RC + dt);
28     }
29
30     /**
31      * 滤波计算
32      */
33     float apply(float input)
34     {
35         if (!initialized) {
36             output = input;
37             initialized = true;
38         }
39         output = alpha * input + (1 - alpha) * output;
40         return output;
41     }
42

```

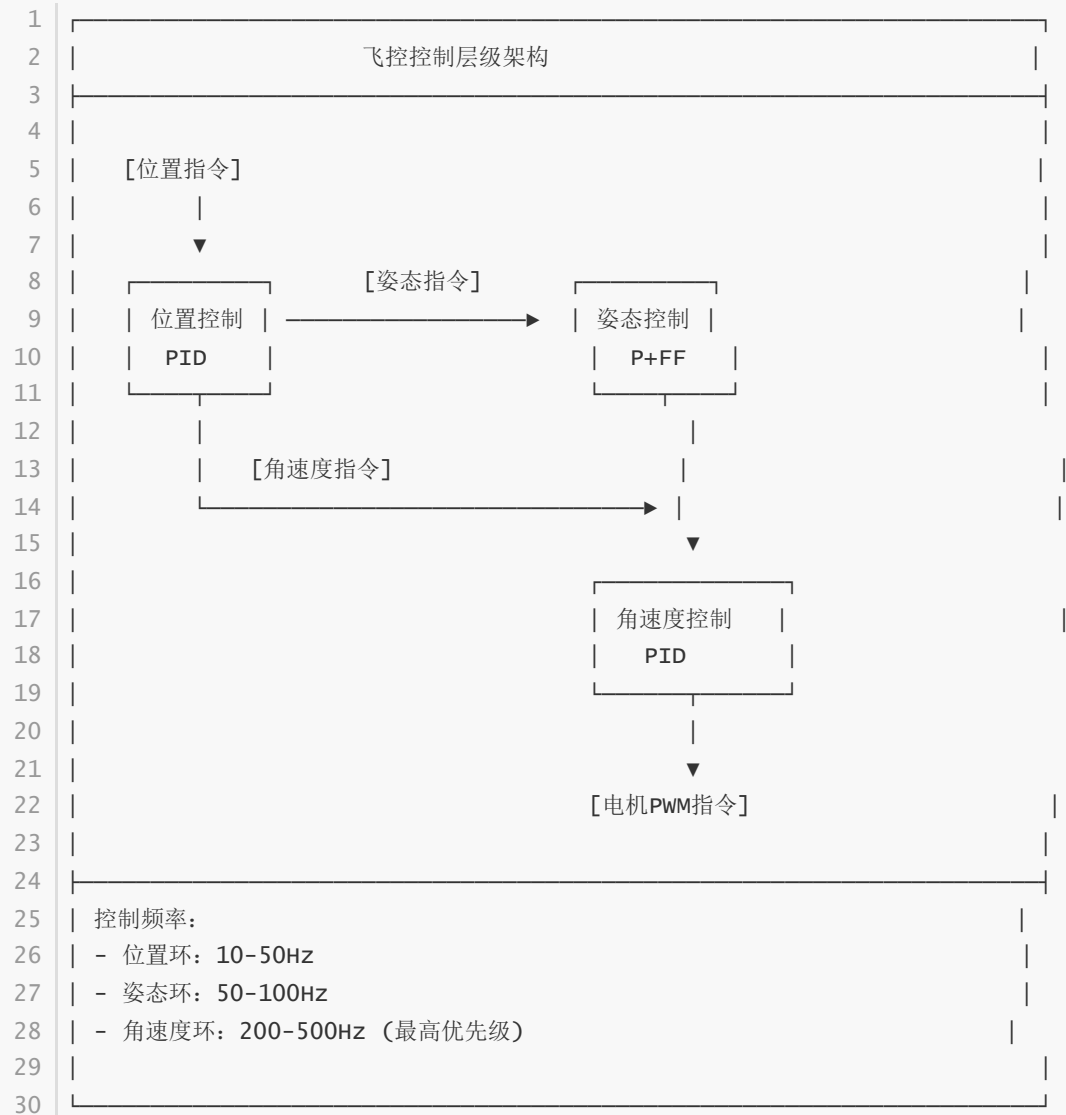
```

43     void reset()
44     {
45         initialized = false;
46         output = 0;
47     }
48 };

```

6.5.6 角速度控制器完整实现

6.5.6.1 姿态控制层级



6.5.6.2 ArduPilot姿态控制代码

```

1  /**
2   * AC_AttitudeControl - 姿态控制器实现
3   *
4   * 文件位置：libraries/AC_AttitudeControl/AC_AttitudeControl.cpp
5   *
6   * 控制流程：
7   * 1. 输入姿态指令 → 计算目标角速度 → PID控制 → 电机混控
8   */
9
10 #include <AC_AttitudeControl.h>

```

```

11 #include <AP_Math/AP_Math.h>
12
13 /**
14  * 设置目标姿态（四元数）
15  *
16  * @param target_attitude 目标姿态四元数
17  *
18  * 数学处理：
19  * 1. 计算当前姿态与目标姿态的四元数差
20  * 2. 将四元数差转换为欧拉角误差
21  * 3. 应用P控制器计算目标角速度
22  */
23 void AC_AttitudeControl::set_attitude_target_quaternion(
24     const Quaternion& target_attitude)
25 {
26     // 获取当前姿态四元数
27     Quaternion current_attitude = get_quaternion();
28
29     // 计算姿态误差四元数
30     //  $q\_error = q\_current^{-1} \otimes q\_target$ 
31     Quaternion q_error = current_attitude.inverse() * target_attitude;
32
33     // 转换为欧拉角误差（小角度近似）
34     Vector3f euler_error = q_error.to_euler();
35
36     /**
37      * 前馈控制（Feedforward）
38      *
39      * 目的：预测目标角速度，提前做出响应
40      *
41      * 对于恒定角速度机动：
42      * - P控制器只响应姿态误差
43      * - 前馈直接提供所需的角速度
44      * - 减小跟踪误差
45      */
46
47     // 计算目标角速度（度/秒）
48     _angvel_target.x = euler_error.x * _p_angle_gain.x + get_roll_rate_ef();
49     _angvel_target.y = euler_error.y * _p_angle_gain.y +
50     get_pitch_rate_ef();
51     _angvel_target.z = euler_error.z * _p_angle_gain.z + get_yaw_rate_ef();
52 }
53
54 /**
55  * 执行角速度控制
56  *
57  * @description
58  * 这是rate环的核心函数，在每帧fast_loop中调用
59  *
60  * 数学流程：
61  * 1. 获取当前角速度测量值
62  * 2. 计算角速度误差
63  * 3. 应用PID控制器
64  * 4. 输出到电机混控
65  */
66 void AC_AttitudeControl::rate_controller_run()

```

```

66 {
67     // 获取当前角速度（从陀螺仪）
68     vector3f rate_meas = _ahrs.get_gyro();
69
70     // ===== Roll角速度环 =====
71     float error_r = _angvel_target.x - rate_meas.x;
72     float output_r = _pid_rate_roll.get_pid_angvel(error_r, rate_meas.x);
73
74     // ===== Pitch角速度环 =====
75     float error_p = _angvel_target.y - rate_meas.y;
76     float output_p = _pid_rate_pitch.get_pid_angvel(error_p, rate_meas.y);
77
78     // ===== Yaw角速度环 =====
79     float error_y = _angvel_target.z - rate_meas.z;
80     float output_y = _pid_rate_yaw.get_pid_angvel(error_y, rate_meas.z);
81
82     // ===== 电机混控 =====
83     /**
84      * 混控公式（四旋翼X模式）：
85      *
86      * motor_FL（前左） = throttle + roll + pitch - yaw    (CW)
87      * motor_FR（前右） = throttle - roll + pitch + yaw    (CCW)
88      * motor_BL（后左） = throttle - roll - pitch - yaw    (CCW)
89      * motor_BR（后右） = throttle + roll - pitch + yaw    (CW)
90      *
91      * 注意符号约定：
92      * - Roll+: 向右滚转（右侧电机加速）
93      * - Pitch+: 向前俯仰（后部电机加速）
94      * - Yaw+: 顺时针旋转（CCW电机加速产生反扭矩）
95      */
96
97     // 各通道范围限制
98     output_r = constrain_float(output_r, -1000, 1000);
99     output_p = constrain_float(output_p, -1000, 1000);
100    output_y = constrain_float(output_y, -1000, 1000);
101
102    // 电机混控
103    _motors.output = _hover_throttle;
104
105    _motors.motor_out[0] = _hover_throttle + output_r + output_p - output_y;
106    // FL
107    _motors.motor_out[1] = _hover_throttle - output_r + output_p + output_y;
108    // FR
109    _motors.motor_out[2] = _hover_throttle - output_r - output_p - output_y;
110    // BL
111    _motors.motor_out[3] = _hover_throttle + output_r - output_p + output_y;
112    // BR
113
114    // PWM输出
115    _motors.output_pwm();
116 }

```

6.5.7 输入整形算法

6.5.7.1 原因与应用

输入整形的目的：

- 抑制系统振荡
- 减少残余振动
- 改善跟踪精度

数学原理：

对于二阶系统：

$$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x = K \cdot u$$

输入整形器由一系列脉冲组成：

$$h(t) = \sum_{i=1}^n A_i \cdot \delta(t - t_i)$$

ZV/ZVD整形器：

- ZV (Zero Vibration)：消除指定模态振动
- ZVD (Zero Vibration Derivative)：对参数变化更鲁棒

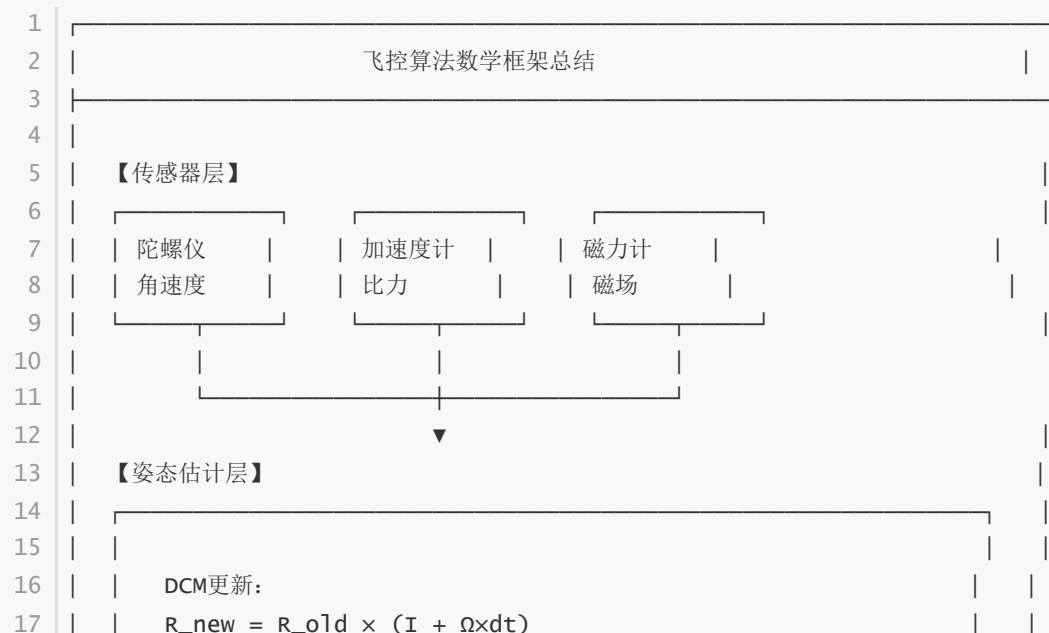
```
1  /**
2   * 输入整形器实现
3   *
4   * ZV整形器参数：
5   * A = [1, exp(-zeta*pi/sqrt(1-zeta^2))]
6   * t = [0, pi/omega_d]
7   *
8   * 其中阻尼比 zeta = cos(phi), omega_d = omega_n*sin(phi)
9   */
10 class InputShaper {
11 private:
12     float amplitude;          // 整形器增益
13     float period;             // 振荡周期
14     float damping;            // 阻尼比
15     float last_impulse_time;  // 上次脉冲时间
16     float impulse_sum;        // 脉冲累积
17
18 public:
19     InputShaper() : last_impulse_time(0), impulse_sum(0) {}
20
21     /**
22      * 设置整形器参数
23      *
24      * @param zeta      阻尼比
25      * @param wn         自然频率 (rad/s)
26      */
27     void set_parameters(float zeta, float wn)
28     {
29         damping = zeta;
30         period = 2 * M_PI / (wn * sqrt(1 - zeta * zeta));
31
32         // ZV整形器：两个脉冲
33         amplitude = 1.0f / (1 + exp(-zeta * M_PI / sqrt(1 - zeta * zeta)));
```

```

34     }
35
36     /**
37     * 应用输入整形
38     *
39     * @param input    原始输入指令
40     * @param dt        时间步长
41     * @return          整形后的输入
42     */
43     float apply(float input, float dt)
44     {
45         float output = 0;
46
47         // 检测输入变化（脉冲）
48         if (fabs(input - last_input) > threshold) {
49             // 生成ZV整形脉冲
50             impulse_sum = input * amplitude;
51             output = impulse_sum;
52             last_impulse_time = 0;
53         } else {
54             // 累积衰减振荡
55             if (impulse_sum > 0.001f) {
56                 float decay = exp(-damping * last_impulse_time / period);
57                 impulse_sum = impulse_sum * decay;
58                 output = impulse_sum;
59             }
60         }
61
62         last_impulse_time += dt;
63         last_input = input;
64
65         return output;
66     }
67 };

```

6.5.8 总结：飞控算法数学框架



7.1.1 参数类型枚举

```
1 enum ap_var_type {
2     AP_PARAM_NONE      = 0,      // 无效类型
3     AP_PARAM_INT8       = 1,      // 8位整数 (-128 ~ 127)
4     AP_PARAM_INT16      = 2,      // 16位整数 (-32768 ~ 32767)
5     AP_PARAM_INT32      = 3,      // 32位整数
6     AP_PARAM_FLOAT      = 4,      // 单精度浮点数
7     AP_PARAM_VECTOR3F   = 5,      // 3维向量 (3个float)
8     AP_PARAM_GROUP      = 6,      // 参数组 (容器)
9 };
```

7.1.2 参数类模板定义

```
1 // 参数类模板定义 (在AP_Param.h中)
2 typedef AP_ParamT<int8_t, AP_PARAM_INT8>    AP_Int8;
3 typedef AP_ParamT<int16_t, AP_PARAM_INT16>  AP_Int16;
4 typedef AP_ParamT<int32_t, AP_PARAM_INT32>  AP_Int32;
5 typedef AP_ParamT<float, AP_PARAM_FLOAT>    AP_Float;
6 typedef AP_ParamV<Vector3f, AP_PARAM_VECTOR3F> AP_Vector3f;
```

7.1.3 创建自定义参数组

```
1 /**
2  * 创建自定义参数组的步骤:
3  * 1. 在类中声明参数成员
4  * 2. 定义var_info表描述参数结构
5  * 3. 在飞控初始化中注册
6  */
7
8 // ===== 步骤1: 声明参数成员 =====
9
10 class MyModule {
11 public:
12     // 整数参数
13     AP_Int8 my_mode;           // 模式选择
14     AP_Int16 my_count;         // 计数器
15     AP_Int16 my_delay;         // 延时参数
16
17     // 浮点参数
18     AP_Float my_speed;         // 速度参数
19     AP_Float my_gain;          // 增益参数
20
21     // 向量参数
22     AP_Vector3f my_offset;     // 偏移量
23
24     /**
25      * 参数组信息表
26      * 这是一个静态常量数组, 告诉参数系统有哪些参数
27      */
28     static const struct AP_Param::GroupInfo var_info[];
29 };
30
31 // ===== 步骤2: 定义var_info表 =====
```

```

32
33 const struct AP_Param::GroupInfo MyModule::var_info[] = {
34     /**
35      * AP_GROUPINFO宏说明:
36      * - name: 参数名称 (不含前缀, 如"MODE")
37      * - idx: 参数索引 (同一组内唯一, 0-63)
38      * - clazz: 所属类名
39      * - element: 类中的成员变量名
40      * - def: 默认值
41      *
42      * 最终参数名 = MyModule + "_" + name = "MYMODULE_MODE"
43      */
44     AP_GROUPINFO("MODE", 0, MyModule, my_mode, 0), // 默认模式0
45     AP_GROUPINFO("COUNT", 1, MyModule, my_count, 10), // 默认10
46     AP_GROUPINFO("DELAY", 2, MyModule, my_delay, 100), // 默认100ms
47     AP_GROUPINFO("SPEED", 3, MyModule, my_speed, 1.5f), // 默认1.5
48     AP_GROUPINFO("GAIN", 4, MyModule, my_gain, 0.5f), // 默认0.5
49     AP_GROUPINFO("OFFSET", 5, MyModule, my_offset, 0.0f), // 默认(0,0,0)
50
51     // 结束标记 (必须!)
52     AP_GROUPEND
53 };
54
55 // ===== 步骤3: 使用参数 =====
56
57 void MyModule::update() {
58     // 读取参数值
59     int8_t mode = my_mode.get(); // 获取整数参数
60     float speed = my_speed.get(); // 获取浮点参数
61     Vector3f offset = my_offset.get(); // 获取向量参数
62
63     // 设置参数值
64     my_speed.set(2.0f); // 设置新值
65
66     // 使用参数
67     if (mode == 0) {
68         // 模式0的处理
69     }
70 }

```

7.1.4 参数访问方法

```

1 // ===== 1. 直接访问 =====
2
3 void direct_access_example(MyModule &module) {
4     // 读取
5     int8_t value = module.my_mode.get();
6     float speed = module.my_speed.get();
7
8     // 写入
9     module.my_speed.set(2.5f);
10
11     // 保存到EEPROM
12     module.my_speed.save();
13 }

```

```

14
15 // ===== 2. 按名称查找 =====
16
17 void find_by_name_example() {
18     // 通过名称查找参数
19     AP_Float *speed_param = (AP_Float *)AP_Param::find("MYMODULE_SPEED");
20
21     if (speed_param != nullptr) {
22         float speed = speed_param->get();
23         speed_param->set(3.0f);
24         speed_param->save();
25     } else {
26         // 参数不存在
27     }
28 }
29
30 // ===== 3. 保存所有参数 =====
31
32 void save_all_params() {
33     // 异步保存所有参数
34     AP_Param::flush();
35
36     // 同步保存（阻塞）
37     AP_Param::flush();
38 }
39
40 // ===== 4. 重置为默认值 =====
41
42 void reset_to_default() {
43     module.my_speed.set_default(1.5f);
44     module.my_speed.save();
45 }

```

7.1.5 高级参数定义

```

1 // ===== 1. 子组（嵌套参数组） =====
2
3 class ParentClass {
4 public:
5     AP_Int8 parent_param;
6     MyModule child;          // 子模块
7
8     static const AP_Param::GroupInfo var_info[];
9 };
10
11 const AP_Param::GroupInfo ParentClass::var_info[] = {
12     AP_GROUPINFO("PARENT", 0, ParentClass, parent_param, 0),
13
14     // 使用SUBGROUPINFO嵌套子模块参数
15     // 最终名称: PARENT_CHILD_MODE, PARENT_CHILD_SPEED 等
16     AP_SUBGROUPINFO(child, "CHILD", 1, ParentClass, MyModule),
17
18     AP_GROUPEND
19 };
20

```

```

21 // ===== 2. 条件编译参数 =====
22
23 // 使用#if控制参数是否存在
24 #if MY_MODULE_ENABLED == 1
25 class MyModule {
26 public:
27     AP_Int8 my_param;
28     static const AP_Param::GroupInfo var_info[];
29 };
30
31 const AP_Param::GroupInfo MyModule::var_info[] = {
32     AP_GROUPINFO("PARAM", 0, MyModule, my_param, 0),
33     AP_GROUPEND
34 };
35 #endif
36
37 // ===== 3. 帧类型参数 =====
38
39 // 只在特定飞行器类型显示的参数
40 #define AP_PARAM_FRAME_COPTER (1<<0) // 多旋翼
41 #define AP_PARAM_FRAME_PLANE (1<<2) // 固定翼
42 #define AP_PARAM_FRAME_ROVER (1<<1) // 无人车
43
44 // 定义只在多旋翼显示的参数
45 AP_GROUPINFO_FRAME("COPTER_ONLY", 0, MyClass, my_param, 0,
46                     AP_PARAM_FRAME_COPTER)

```

7.1.6 常见参数配置

```

1 // 多旋翼常用参数示例:
2
3 // 电机极对数（电调设置）
4 AP_Int8 MOTOR_PWM_TYPE; // PWM类型: 0=普通, 1=OneShot125, 2=OneShot42
5
6 // PID参数
7 AP_Float ATC_RATE_RP_P; // 姿态P增益
8 AP_Float ATC_RATE_RP_I; // 姿态I增益
9 AP_Float ATC_RATE_RP_D; // 姿态D增益
10 AP_Float ATC_RATE_Y_P; // 偏航P增益
11 AP_Float ATC_RATE_Y_I; // 偏航I增益
12 AP_Float ATC_RATE_Y_D; // 偏航D增益
13
14 // 滤波器参数
15 AP_Float INS_GYRO_FILTER; // 陀螺仪滤波器截止频率 (Hz)
16 AP_Float INS_ACCEL_FILTER; // 加速度计滤波器截止频率 (Hz)
17
18 // 飞行模式
19 AP_Int8 FLTMODE1; // 通道1的飞行模式
20 AP_Int8 FLTMODE2; // 通道2的飞行模式
21 AP_Int8 FLTMODE3; // 通道3的飞行模式
22 AP_Int8 FLTMODE4; // 通道4的飞行模式
23 AP_Int8 FLTMODE5; // 通道5的飞行模式
24 AP_Int8 FLTMODE6; // 通道6的飞行模式
25
26 // RC输入映射

```

```

27 AP_Int8 RCMAP_ROLL;           // 横滚通道映射
28 AP_Int8 RCMAP_PITCH;         // 俯仰通道映射
29 AP_Int8 RCMAP_THROTTLE;      // 油门通道映射
30 AP_Int8 RCMAP_YAW;           // 偏航通道映射

```

7.2 参数框架原理

```

1  参数存储原理：
2
3  1. 【定义阶段】
4      - 类定义时声明AP_Int8/AP_Float等参数成员
5      - 创建var_info表描述参数结构
6      - 每个参数被分配唯一的key值
7
8  2. 【初始化阶段】
9      - AP_Param::setup() 扫描所有var_info表
10     - 为每个参数分配唯一的索引(key)
11     - 从EEPROM加载保存的值
12     - 如果EEPROM中没有，则使用默认值
13
14  3. 【使用阶段】
15     - 通过参数对象直接访问
16     - 通过AP_Param::find()按名称查找
17     - MAVLink可以读写参数
18
19  4. 【保存阶段】
20     - 参数.set_and_save() 保存单个参数
21     - AP_Param::flush() 保存所有参数
22     - 数据写入EEPROM/Flash
23
24  参数命名空间：
25     - 全局参数： "PARAM_NAME"
26     - 参数组内： "GROUP_PARAM_NAME"
27     - 嵌套参数： "PARENT_CHILD_NAME"
28
29  实际参数名示例：
30     - INS_GYRO_FILTER           (INS库的GYRO_FILTER参数)
31     - SCHED_LOOP_RATE          (SCHED库的LOOP_RATE参数)
32     - RC1_MIN                   (RC通道1的最小值)
33     - ATC_RATE_RP_P            (姿态控制的roll/pitch P增益)

```

8. 通信系统

8.1 GCS_MAVLink - 地面站通信

文件位置: `libraries/GCS_MAVLink/`

MAVLink是无人机通信的**标准协议**，用于飞控与地面站之间的通信。

8.1.1 MAVLink消息类型

```
1 // 常用MAVLink消息ID
2 enum {
3     MAVLINK_MSG_ID_HEARTBEAT           = 0,      // 心跳包
4     MAVLINK_MSG_ID_SYS_STATUS          = 1,      // 系统状态
5     MAVLINK_MSG_ID_ATTITUDE            = 30,      // 姿态数据
6     MAVLINK_MSG_ID_GPS_RAW_INT         = 24,      // GPS原始数据
7     MAVLINK_MSG_ID_RC_CHANNELS          = 65,      // RC通道数据
8     MAVLINK_MSG_ID_MISSION_ITEM         = 39,      // 航点数据
9     MAVLINK_MSG_ID_PARAM_VALUE          = 22,      // 参数值
10    MAVLINK_MSG_ID_COMMAND_ACK           = 77,      // 命令确认
11    MAVLINK_MSG_ID_STATUSTEXT            = 253,     // 状态文本
12 };
```

8.1.2 发送数据到地面站

```
1 // ===== 1. 发送姿态数据 =====
2
3 void send_attitude_example(GCS_MAVLINK &gcs) {
4     // 自动发送（系统周期性发送）
5     // 无需手动调用
6 }
7
8 // ===== 2. 发送状态文本 =====
9
10 void send_statustext_example() {
11     // 发送普通信息（绿色文字）
12     GCS_SEND_TEXT(MAV_SEVERITY_INFO, "Hello from ArduPilot!");
13
14     // 发送警告（黄色文字）
15     GCS_SEND_TEXT(MAV_SEVERITY_WARNING, "Low battery!");
16
17     // 发送错误（红色文字）
18     GCS_SEND_TEXT(MAV_SEVERITY_ERROR, "Sensor error!");
19
20     // 发送调试信息（灰色文字，不显示）
21     GCS_SEND_TEXT(MAV_SEVERITY_DEBUG, "Debug: value=%d", value);
22 }
23
24 /**
25  * MAV_SEVERITY 级别:
26  * 0 = Emergency      - 紧急（红色闪烁）
27  * 1 = Alert          - 警报（红色）
28  * 2 = Critical        - 严重（红色）
29  * 3 = Error           - 错误（橙色）
30  * 4 = Warning         - 警告（黄色）
31  * 5 = Notice          - 通知（淡黄）
32  * 6 = Informational  - 信息（白色）
33  * 7 = Debug           - 调试（灰色）
34  */
```

8.2 AP_SerialManager - 串口管理

文件位置: `libraries/AP_SerialManager/`

管理多个串口的配置和协议分配。

8.2.1 串口协议枚举

```
1 enum SerialProtocol {
2     NONE           = 0,
3     MAVLINK        = 1,      // MAVLink地面站
4     GPS             = 2,      // GPS接收机
5     GPS_MOVEBEACON = 3,      // 移动信标
6     ADSB           = 5,      // ADS-B接收机
7     LIGHTWARE_SF    = 7,      // 光流传感器
8     FRSKY_D         = 9,      // FRSky D协议
9     FRSKY_SPORT     = 10,     // FRSky S.Port
10    HOTT            = 11,     // HoTT遥测
11    MSP             = 12,     // MSP (Multiwii Serial Protocol)
12    DJI_FPV         = 13,     // DJI FPV
13    // ... 更多
14 };
```

8.2.2 串口配置参数

```
1 常用串口参数:
2  - SERIALn_PROTOCOL: 串口协议 (MAVLink=1, GPS=2等)
3  - SERIALn_BAUD:     波特率 (115200, 57600, 9600等)
4  - SERIALn_OPTIONS:  串口选项
5
6 示例配置 (SERIAL0 = USB/调试串口):
7  - SERIAL0_PROTOCOL = 1 (MAVLink)
8  - SERIAL0_BAUD = 115200
9
10 示例配置 (SERIAL1 = GPS串口):
11 - SERIAL1_PROTOCOL = 2 (GPS)
12 - SERIAL1_BAUD = 38400
```

8.2.3 使用示例

```
1 // ===== 1. 获取MAVLink串口 =====
2
3 void mavlink_example() {
4     AP_SerialManager *serial = AP::serialmanager();
5
6     // 获取MAVLink串口
7     AP_HAL::UARTDriver *uart = serial->
>find_serial(AP_SerialManager::SerialProtocol::MAVLINK);
8
9     if (uart) {
10         // 发送数据
11         uart->printf("Hello MAVLink!\n");
12
13         // 检查可用数据
```

```

14         if (uart->available() > 0) {
15             uint8_t c = uart->read();
16             // 处理接收到的字节
17         }
18     }
19 }
20
21 // ===== 2. 获取GPS串口 =====
22
23 void gps_serial_example() {
24     AP_SerialManager *serial = AP::serialmanager();
25
26     AP_HAL::UARTDriver *gps_uart = serial->
>find_serial(AP_SerialManager::SerialProtocol::GPS);
27
28     if (gps_uart) {
29         // GPS串口用于连接GPS模块
30         // ArduPilot自动处理GPS协议解析
31     }
32 }

```

9. 飞行器类型

9.1 ArduCopter - 多旋翼飞控

文件位置: `ArduCopter/`

9.1.1 Copter类结构

```

1 class Copter : public AP_Vehicle {
2 private:
3     // ===== 核心组件 =====
4     AP_MultiCopter aparm;                // 多旋翼参数
5     AC_AttitudeControl *attitude_control; // 姿态控制
6     AC_PosControl *pos_control;           // 位置控制
7     AC_WPNav *wp_nav;                     // 航点导航
8     AC_Loiter *loiter_nav;                // 悬停导航
9
10    // ===== 状态 =====
11    Mode *flightmode;                      // 当前飞行模式
12    AP_BattMonitor battery;                 // 电池监控
13
14    // ===== RC输入 =====
15    RC_Channel *channel_roll;               // 横滚通道
16    RC_Channel *channel_pitch;              // 俯仰通道
17    RC_Channel *channel_throttle;           // 油门通道
18    RC_Channel *channel_yaw;                // 偏航通道
19
20    // ===== 传感器 =====
21    AP_RangeFinder rangefinder;              // 距离传感器
22    AP_OpticalFlow optflow;                 // 光流传感器
23
24    // ===== 任务列表 =====
25    static const AP_Scheduler::Task scheduler_tasks[];

```


9.1.2 初始化流程

```
1 void Copter::setup() {
2     // 1. HAL初始化
3     AP_HAL::HAL::setup();
4
5     // 2. 加载参数
6     load_parameters();
7
8     // 3. 初始化传感器
9     ins.init(1000);           // IMU, 采样率1000Hz
10    compass.init();           // 磁力计
11    barometer.init();          // 气压计
12    gps.init();                // GPS
13
14    // 4. 初始化电机
15    motors->init();
16
17    // 5. 初始化RC输入
18    init_rc_in();
19
20    // 6. 初始化飞控特定组件
21    init_ardupilot();
22
23    // 7. 初始化任务调度
24    AP_Scheduler::init(scheduler_tasks, ARRAY_SIZE(scheduler_tasks));
25 }
26
27 void Copter::loop() {
28     // 调用调度器循环
29     AP_Scheduler::loop();
30 }
31
32 void Copter::fast_loop() {
33     // 1. 读取传感器
34     read_inertia();
35     update_GPS();
36     read_rangefinder();
37
38     // 2. 更新AHRS
39     update_AHRS();
40
41     // 3. 更新飞行模式
42     update_flight_mode();
43
44     // 4. 执行控制
45     attitude_control->rate_controller_run();
46
47     // 5. 输出到电机
48     motors_output();
49 }
```

9.2 ArduPlane - 固定翼飞控

文件位置: ArduPlane/

9.2.1 Plane类结构

```
1 class Plane : public AP_Vehicle {
2 private:
3     // ===== 控制器 =====
4     AP_RollController rollController;    // 横滚控制器
5     AP_PitchController pitchController;  // 俯仰控制器
6     AP_YawController yawController;      // 偏航控制器
7     AP_TECS tecs;                        // 总能量控制
8     AP_L1_Control l1_controller;         // L1导航控制器
9
10    // ===== 参数 =====
11    AP_FixedWing aparm;
12
13    // ===== RC输入 =====
14    RC_Channel *channel_roll;
15    RC_Channel *channel_pitch;
16    RC_Channel *channel_throttle;
17    RC_Channel *channel_rudder;
18 };
```

9.3 Rover - 无人车

文件位置: Rover/

```
1 class Rover : public AP_Vehicle {
2 private:
3     // ===== 控制器 =====
4     AP_SteerController steer_controller; // 转向控制器
5     AR_PosControl pos_control;           // 位置控制
6
7     // ===== 车辆特定 =====
8     AP_Vehicle::DiffThrottleControl diff_throttle; // 差速控制
9 };
```

10. 学习路径与实践建议

10.1 推荐学习顺序

- 1 第1周: 基础架构
- 2 |— 1.1 理解项目结构
- 3 | |— 阅读本文档第1-2章
- 4 |— 1.2 HAL层概念
- 5 | |— libraries/AP_HAL/ 相关头文件
- 6 |— 1.3 任务调度器
- 7 | |— libraries/AP_Scheduler/
- 8 |— 1.4 参数系统
- 9 | |— libraries/AP_Param/

```
10 └─ 1.5 编译和运行SITL仿真
11     └─ ./Tools/autotest/sim_vehicle.py
12
13 第2周：传感器与姿态
14 └─ 2.1 IMU数据读取
15     └─ libraries/AP_InertialSensor/
16 └─ 2.2 GPS数据读取
17     └─ libraries/AP_GPS/
18 └─ 2.3 AHRS姿态估计
19     └─ libraries/AP_AHRS/
20 └─ 2.4 DCM算法
21     └─ AP_AHRS_DCM.cpp
22 └─ 2.5 EKF3原理
23     └─ libraries/AP_NaveKF3/
24
25 第3周：控制系统
26 └─ 3.1 PID控制原理
27     └─ libraries/PID/
28 └─ 3.2 姿态控制实现
29     └─ libraries/AC_AttitudeControl/
30 └─ 3.3 位置控制实现
31     └─ AC_PosControl.cpp
32 └─ 3.4 调参实践
33     └─ Mission Planner / QGroundControl
34 └─ 3.5 飞行模式
35     └─ ArduCopter/mode*.cpp
36
37 第4周：通信与任务
38 └─ 4.1 MAVLink协议
39     └─ libraries/GCS_MAVLink/
40 └─ 4.2 地面站通信
41     └─ Mission Planner连接测试
42 └─ 4.3 任务规划
43     └─ AP_Mission/
44 └─ 4.4 安全机制
45     └─ AP_Arming/, failsafe相关
```

10.2 SITL仿真环境搭建

```
1 # Linux系统安装依赖
2 sudo apt-get update
3 sudo apt-get install python3-pip python3-matplotlib
4 sudo apt-get install g++ g++-multilib git
5
6 # 克隆并初始化子模块
7 git clone https://github.com/ArduPilot/ardupilot.git
8 cd ardupilot
9 git submodule update --init --recursive
10
11 # 运行多旋翼仿真
12 ./Tools/autotest/sim_vehicle.py -v ArduCopter --console --map
13
14 # 常用参数：
15 # -v ArduCopter      : 多旋翼
16 # -v ArduPlane       : 固定翼
```

```
17 # -v Rover      : 无人车
18 # --console     : 显示控制台
19 # --map         : 显示地图
```

10.3 添加新参数实践

```
1 // 文件: libraries/AP_MyModule/MyModule.h
2
3 #pragma once
4
5 #include <AP_Param/AP_Param.h>
6
7 class AP_MyModule {
8 public:
9     AP_MyModule();
10
11     void update();
12
13     // ===== 参数定义 =====
14     AP_Int8 my_mode;           // 模式参数
15     AP_Float my_speed;         // 速度参数
16     AP_Float my_gain;          // 增益参数
17
18     // ===== 参数组信息表 =====
19     static const struct AP_Param::GroupInfo var_info[];
20 };
21
22 // 文件: libraries/AP_MyModule/MyModule.cpp
23
24 #include "MyModule.h"
25
26 AP_MyModule::AP_MyModule() {
27     // 参数注册 (在构造函数中自动完成)
28 }
29
30 const struct AP_Param::GroupInfo AP_MyModule::var_info[] = {
31     AP_GROUPINFO("MODE", 0, AP_MyModule, my_mode, 0),
32     AP_GROUPINFO("SPEED", 1, AP_MyModule, my_speed, 1.0f),
33     AP_GROUPINFO("GAIN", 2, AP_MyModule, my_gain, 0.5f),
34     AP_GROUPEND
35 };
36
37 void AP_MyModule::update() {
38     // 使用参数
39     if (my_mode.get() == 0) {
40         float output = my_speed.get() * my_gain.get();
41         // ...
42     }
43 }
44
45 // 在飞控中使用
46 AP_MyModule my_module;
47
48 void copter_init() {
49     // 无需额外初始化, AP_Param自动扫描var_info表
```

```

50 }
51
52 void copter_update() {
53     my_module.update();
54 }

```

10.4 添加新传感器驱动模板

```

1 // 文件: libraries/AP_MySensor/AP_MySensor.h
2
3 #pragma once
4
5 #include <AP_HAL/AP_HAL.h>
6
7 // 前向声明后端
8 class AP_MySensor_Backend;
9
10 class AP_MySensor {
11 public:
12     AP_MySensor();
13
14     // 获取数据
15     float get_data() const { return data; }
16     bool healthy() const { return _healthy; }
17
18     // 探测和创建后端
19     static AP_MySensor *probe();
20
21     // 参数
22     static const struct AP_Param::GroupInfo var_info[];
23
24 private:
25     friend class AP_MySensor_Backend;
26
27     float data;
28     bool _healthy;
29
30     // 后端列表
31     AP_MySensor_Backend *_backend;
32 };
33
34 // 文件: libraries/AP_MySensor/AP_MySensor.cpp
35
36 #include "AP_MySensor.h"
37 #include "AP_MySensor_Backend.h"
38
39 AP_MySensor::AP_MySensor() {
40     // 初始化
41     AP_Param::setup_object_defaults(this, var_info);
42 }
43
44 const struct AP_Param::GroupInfo AP_MySensor::var_info[] = {
45     AP_GROUPINFO("DATA", 0, AP_MySensor, data, 0),
46     AP_GROUPEND
47 };

```

```

48
49 // 后端实现
50 AP_MySensor_Backend::AP_MySensor_Backend(AP_MySensor &frontend)
51 : _frontend(frontend) {
52 }
53
54 bool AP_MySensor_Backend::init(AP_HAL::OwnPtr<AP_HAL::Device> dev) {
55     _dev = std::move(dev);
56
57     // 检测传感器
58     if (!_dev->write_register(REG_ID, ID_VALUE)) {
59         return false;
60     }
61
62     return true;
63 }
64
65 void AP_MySensor_Backend::update() {
66     uint8_t value;
67     if (_dev->read_register(REG_DATA, value)) {
68         _frontend.data = value;
69         _frontend._healthy = true;
70     } else {
71         _frontend._healthy = false;
72     }
73 }

```

10.5 调试技巧

```

1 // ===== 1. 控制台调试输出 =====
2
3 void debug_example() {
4     // 使用hal.console输出
5     hal.console->printf("Debug: value=%d\n", value);
6     hal.console->printf("Debug: float=%.2f\n", float_value);
7
8     // 使用GCS发送状态文本
9     GCS_SEND_TEXT(MAV_SEVERITY_DEBUG, "Debug: %s", message);
10 }
11
12 // ===== 2. 性能监控 =====
13
14 void perf_example() {
15     // 获取循环时间
16     float loop_time = AP::scheduler().get_filtered_loop_time();
17
18     // 获取CPU负载
19     float load = AP::scheduler().load_average();
20
21     if (load > 0.8f) {
22         GCS_SEND_TEXT(MAV_SEVERITY_WARNING, "High CPU load: %.1f%%", load *
23         100);
24     }
25 }

```

```

26 // ===== 3. 日志记录 =====
27
28 void logging_example() {
29     // 记录到DataFlash
30     AP::logger().Write("MYDBG", "TimeUS,value", "QF",
31                       AP_HAL::micros64(), value);
32
33     // 记录多个值
34     AP::logger().Write("MYDBG", "TimeUS,v1,v2,v3", "Qfff",
35                       AP_HAL::micros64(), v1, v2, v3);
36 }
37
38 // ===== 4. 条件调试 =====
39
40 void conditional_debug() {
41     // 只在特定条件下输出
42     static uint32_t last_debug = 0;
43     if (AP_HAL::millis() - last_debug > 1000) { // 每秒一次
44         last_debug = AP_HAL::millis();
45         // 输出调试信息
46     }
47 }

```

10.6 常见问题排查

问题	可能原因	解决方法
姿态振荡	PID参数P太大	降低P增益，增加D增益
姿态漂移	IMU校准问题	重新校准加速度计/陀螺仪
无法解锁	安全开关/GPS	检查安全开关，检查GPS定位
GPS不定位	天线连接/遮挡	检查天线连接，放到开阔地带
电机不转	未解锁/模式	解锁飞控，检查飞行模式
断线	遥控器干扰	检查频率，排查干扰源
定位漂移	EKF问题	重置EKF，检查GPS精度
高度跳变	气压计干扰	遮挡气压计，使用GPS辅助

10.7 推荐学习资源

1. 官方文档: <https://ardupilot.org/>
2. 开发者Wiki: <https://ardupilot.org/dev/>
3. 源码仓库: <https://github.com/ArduPilot/ardupilot>
4. 社区论坛: <https://discuss.ardupilot.org/>
5. MAVLink文档: <https://mavlink.io/en/>

结语

ArduPilot是一个高度模块化、设计精良的开源飞控项目。通过本指南的学习，你应该能够：

- 理解架构**：掌握分层架构设计，理解HAL、Scheduler、AHRS等核心组件
- 使用传感器**：能够读取和处理IMU、GPS、气压计、磁力计数据
- 姿态控制**：理解EKF3和DCM姿态估计算法，掌握PID控制器原理
- 配置参数**：能够添加和配置自定义参数
- 开发扩展**：能够添加新的传感器驱动和功能模块

建议按照学习路径逐步深入，先理解整体架构，再深入各个模块的细节实现。最重要的是通过实践（仿真和实飞）来巩固所学知识。

祝学习顺利！

文档生成时间: 2025年

适用于: ArduPilot 飞行控制器